



Mastère Data Engineering et IA

BIG DATA FRAMEWORK

Steve ELANGA

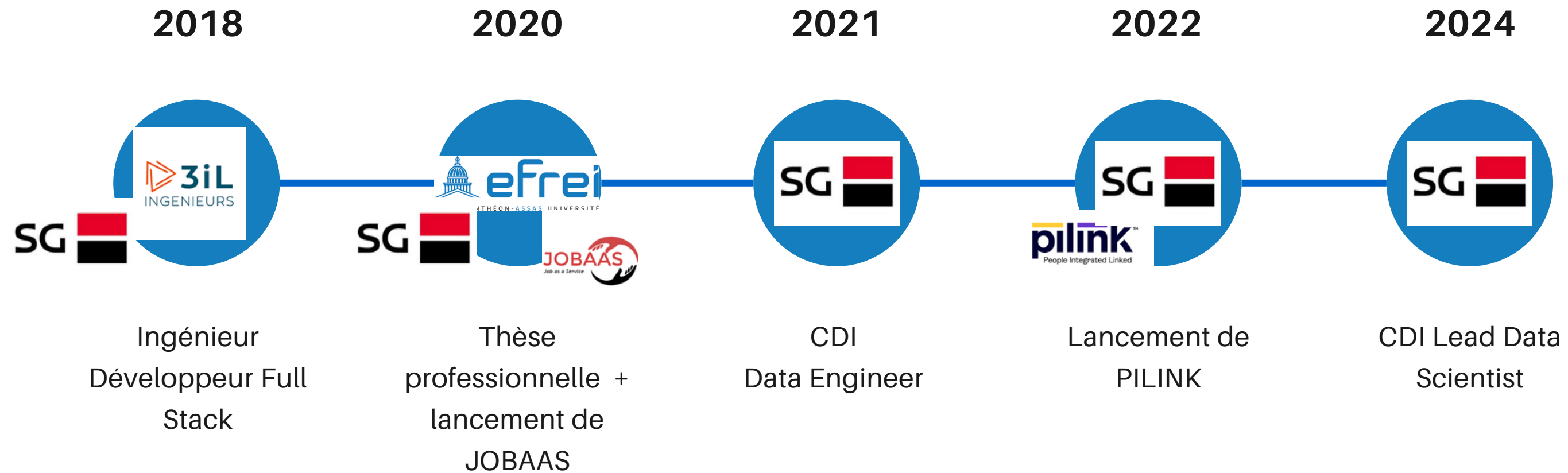
Data Engineer Senior

Année scolaire 2025-2026

HELLO! I'M STEVE

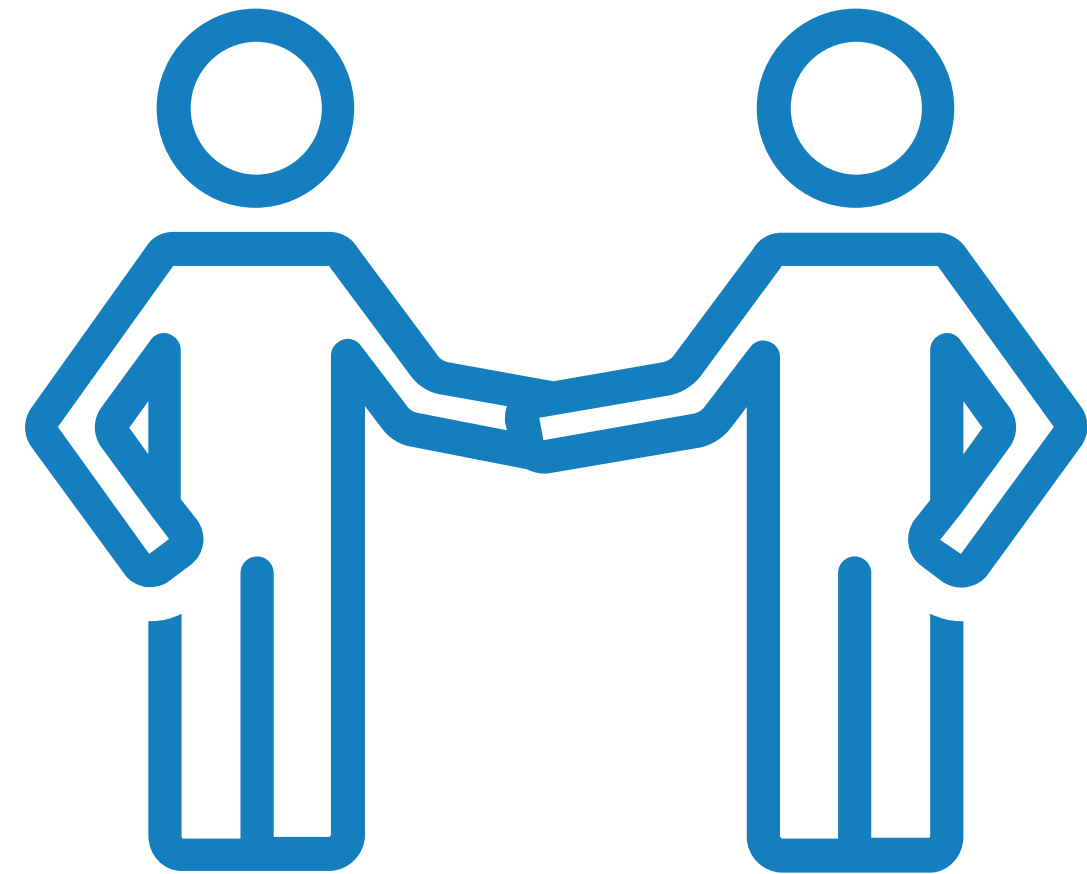
- Origine Camerounaise
- Data Engineer Senior
- Co-fondateur de JOBAAS, Cabinet RH
- Co-fondateur de PILINK
- Mentor Carrière en Data et Dev
- Passionné de danses de salons (Kizomba, Batchata), de piano et de poésie



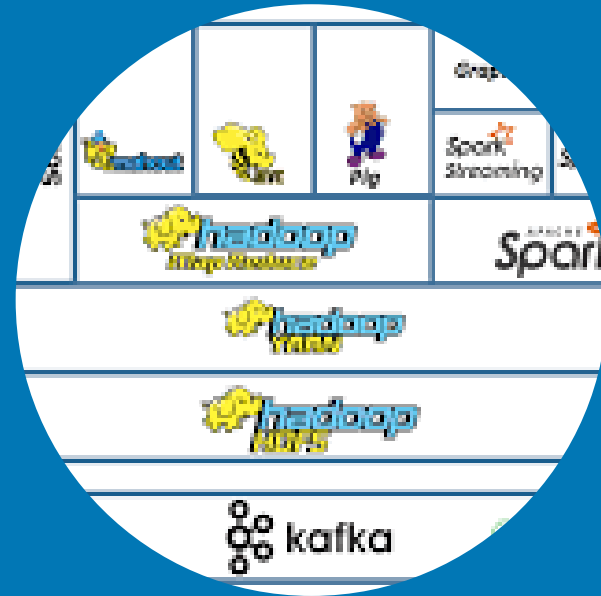


ET VOUS ?

- Nom et Prénom
- Parcours académique en quelques termes
- Objectif professionnel cible
- Attentes de la matière



Agenda



- * **Introduction**
- * **Module 1: Concepts clés du Big Data**
- * **Module 2: Les métiers du Big Data**
- * **Module 3: L'écosystème Hadoop**
- * **Module 4: L'écosystème Spark**
- * **Module 5: Examen Final**

- Comprendre la génèse des métiers de la Data
- Différencier les différents métiers qui entourent la Data
- Comprendre les concepts clés de la Data
- Comprendre l'architecture médaillon d'un Lakehouse
- Comprendre le fonctionnement d'Hadoop
- Comprendre les éléments fondamentaux de Spark
- Combiner Hadoop et Spark dans un projet pour répondre à une problématique métier précise



- Il n'existe pas de questions "bêtes"
- Toute remarque constructive est la bienvenue (sur le cours, les TDs et les TPs)
- Notez vos questions pour la session Q&A
- La classe est une équipe donc pensez à vous entraider
- Tout le monde contribue à l'enrichissement du cours
- Posez la même question plusieurs fois n'est pas un problème, je n'hésiterai pas à me répéter



- Bonus de participation
- Projet sur Spark noté sur 20 (à remettre le dimanche qui suit le dernier cours)
- QCM sur 30 qui comptera pour 40%

- Prouesses technologiques dans le secteur de l'IT
- Production en masse des données
- Avènement des smartphones et objets connectés
- Naissance de plusieurs réseaux sociaux et leur adoption dans le monde
- Transfert de données structurées et non structurées
- Amélioration de l'accessibilité à internet

Abbr.	Prefix name	Decimal size	Size in thousands	Binary approximation	Address variable size
K	kilo-	10^3	1,000	$1,024 = 2^{10}$	10
M	mega-	10^6	$1,000^2$	$1,024^2 = 2^{20}$	20
G	giga-	10^9	$1,000^3$	$1,024^3 = 2^{30}$	30
T	tera-	10^{12}	$1,000^4$	$1,024^4 = 2^{40}$	40
P	peta-	10^{15}	$1,000^5$	$1,024^5 = 2^{50}$	50
E	exa-	10^{18}	$1,000^6$	$1,024^6 = 2^{60}$	60
Z	zetta-	10^{21}	$1,000^7$	$1,024^7 = 2^{70}$	70
Y	yotta-	10^{24}	$1,000^8$	$1,024^8 = 2^{80}$	80

**Tableau des unités
de stockage des données**

Compagnies	Données traitées (2014)	Données stockées (2014)
Google	100 Po	15 000 Po
Ebay	100 Po	90 Po
Facebook	600 To	300 Po
Twitter	100 To	100 To
Baidu	10-100 Po	2 000 Po
NSA	29 Po	10 000 Po

**Volume de données traitées
par quelques entreprises en 2014**

- Comment assurer la collecte de ces données ?
- Comment assurer le stockage de ces données ?
- Comment assurer la distribution de ces données ?
- Comment exploiter ces données afin de créer de la plus value ?

Usage des frameworks Big data

- Production de reportings sur des données à échelle
- Prédire des résultats et faire des stratégies basées sur ces prédictions
- Automatiser les traitements (amélioration de l'efficacité opérationnelle)



“

**La donnée est l’or noir
du 21e siècle**

”



Questions / Réponses



Evaluation du cours

MODULE 1: CONCEPTS CLES DU BIG DATA

01

Les 5V du BIG
DATA

02

Les structures de
données

03

Le format parquet

04

Les systèmes de
stockage BIG
DATA

05

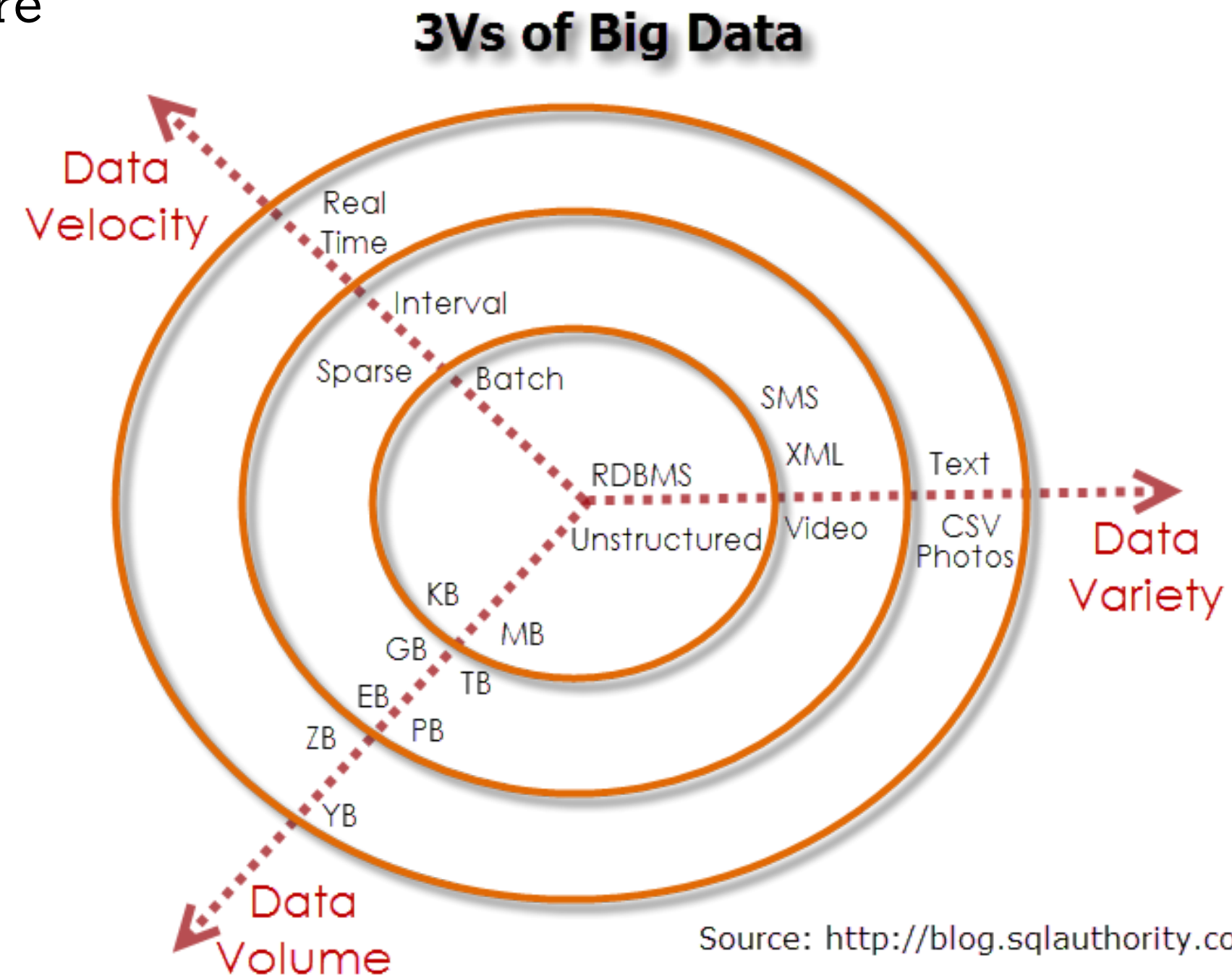
Le Cloud

06

La virtualisation

- Volume: Les systèmes de stockage doivent permettre d'ingérer et de conserver des Gigas de données voir des Péta données.
- Variété: Les données récoltées sont à la fois structurées et non structurées
- Vélocité: Les données peuvent être collectées par lots ou en temps réel (batch, mini-batch, streaming, event trigger)

Deux critères tout aussi important qui font que maintenant on parle de 5V sont: La véracité et la valeur



- Les données structurées: Ce sont des données qui sont formatées et transformées dans un schéma de données bien défini.
- Notion d'identifiant unique , de modèle de données unique pour les données de même nature
- Espace de stockage en mémoire prédéfini
- Flexibilité réduite dans la mise à jour des structures de données
- Optimisation dans la collecte de ces données
- Caractéristique principale: Base de données relationnelles, utilisation du SQL

	A	B	C	D	E	F	
1	LISTE DES ÉLÈVES					titre de la liste	
2	n°	Nom	Prénom	Option	Ville	champs	
3	1	BARDAMU	Sylvia	gestion	Olivet	enregistrements	
4	2	BERNARD	Pie	com	Orléans		
5	3	BERRURIER	Jessy	gestion	Olivet		
6	4	BERTRAND	Hassan	com	Fleury		
7	5	BESNARD	Pleurote	com	Orléans		
8	6	CARRÉ	Alice	com	Bucy		
9	7	COULLON	Arthur	com	Orléans		
10	8	CUGNARD	Johnny	com	Chevilly		
11	9	DEJARDIN	Mohamed	gestion	Orléans		
12	10	EL ALAOUI	Renée	com	Orléans		



- Les données non structurées ou sémi structurées: Ce sont les données brutes, sans schéma prédéfini même quand elles sont de même nature. Exemple: Les mails ou des conversations.
- La taille de ces données en mémoire n'est pas prédéfinie
- Flexibilité sur la structure des données
- Difficulté dans le traitement de ces données
- Caractéristiques principales: Base de données non relationnelles, NoSQL



Le parquet est un format de fichiers qui permet de stocker les données en colonne.

L'objectif est d'assurer le stockage et le requêtage des données de façon efficiente.

Caractéristiques:

- Indépendance aux langages
- Format Colonne
- Adapté aux traitements analytiques
- Haute efficacité pour la compression des données

Avantages:

- Idéal pour stocker tout type de données Big Data
- Optimisation de l'espace de stockage
- Amélioration des performances pour l'analytics



Parquet

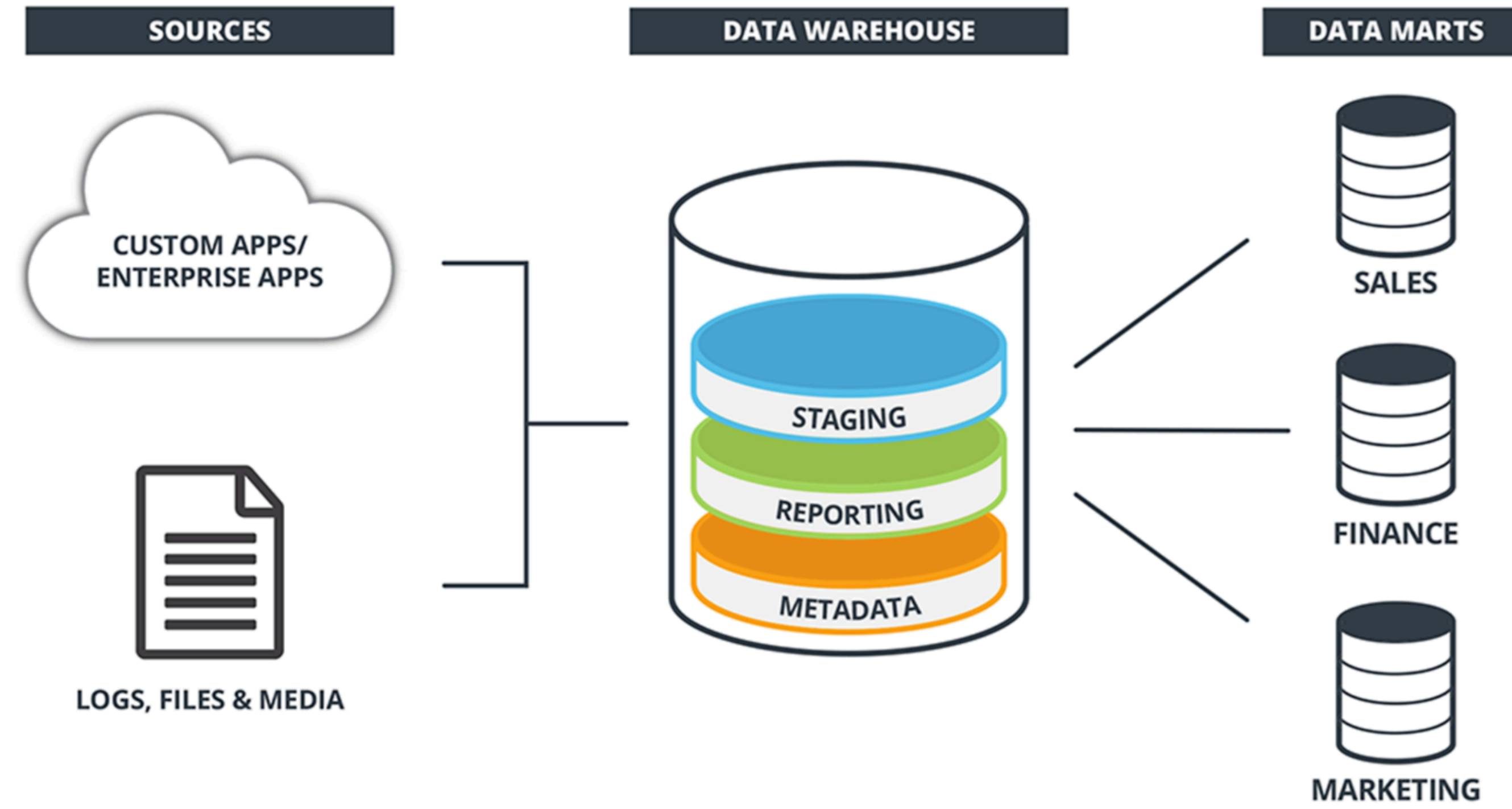
LE FORMAT PARQUET

	Column 1	Column 2	Column 3	Column 4	Column 5
	Product	Customer	Country	Date	Sales Amount
Row Group 1	Ball	John Doe	USA	2023-01-01	100
	T-Shirt	John Doe	USA	2023-01-02	200
Row Group 2	Socks	Maria Adams	UK	2023-01-01	300
	Socks	Antonio Grant	USA	2023-01-03	100
Row Group 3	T-Shirt	Maria Adams	UK	2023-01-02	500
	Socks	John Doe	USA	2023-01-05	200

LES SYSTEMES DE STOCKAGE: DATAWAREHOUSE

- Base de données relationnelles avec des données structurées hébergée dans un Data Center ou dans un Cloud
- Elle contient des données utilisées pour répondre à des fins de reportings et d'analyses
- Système de stockage idéal pour stocker des Datasets (modèles entreprises)
- Les données sont en général historisées
- Les Datamarts sont des portions de datawarehouse pour répondre à des problématiques métiers spécifiques

LES SYSTEMES DE STOCKAGE: DATAWAREHOUSE



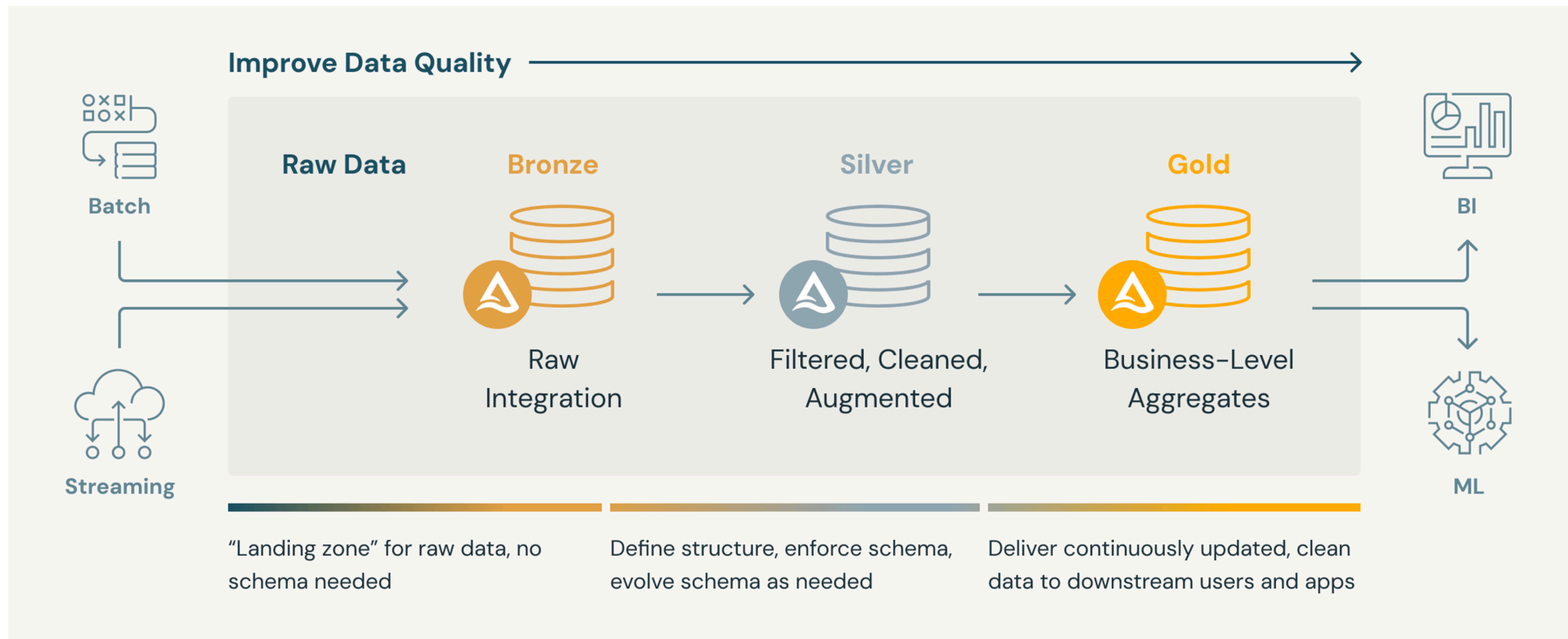
Architecture Datawarehouse

LES SYSTEMES DE STOCKAGE: DATA LAKE

- Système de stockage avec des données structurées et non structurées
- Stockage de données issues de plusieurs sources
- Format de fichier de prédilection: Parquet

Pour optimiser les coûts de stockage, une nouvelle solution combinant le datalake et le datawarehouse est née : Le lakehouse avec son architecture <médaille>. L'avantage c'est d'avoir plusieurs répertoires au sein d'une même couche physique et d'un même environnement technique en fonction du niveau de qualité de la donnée .

ARCHITECTURE MEDAILLON D'UN LAKEHOUSE



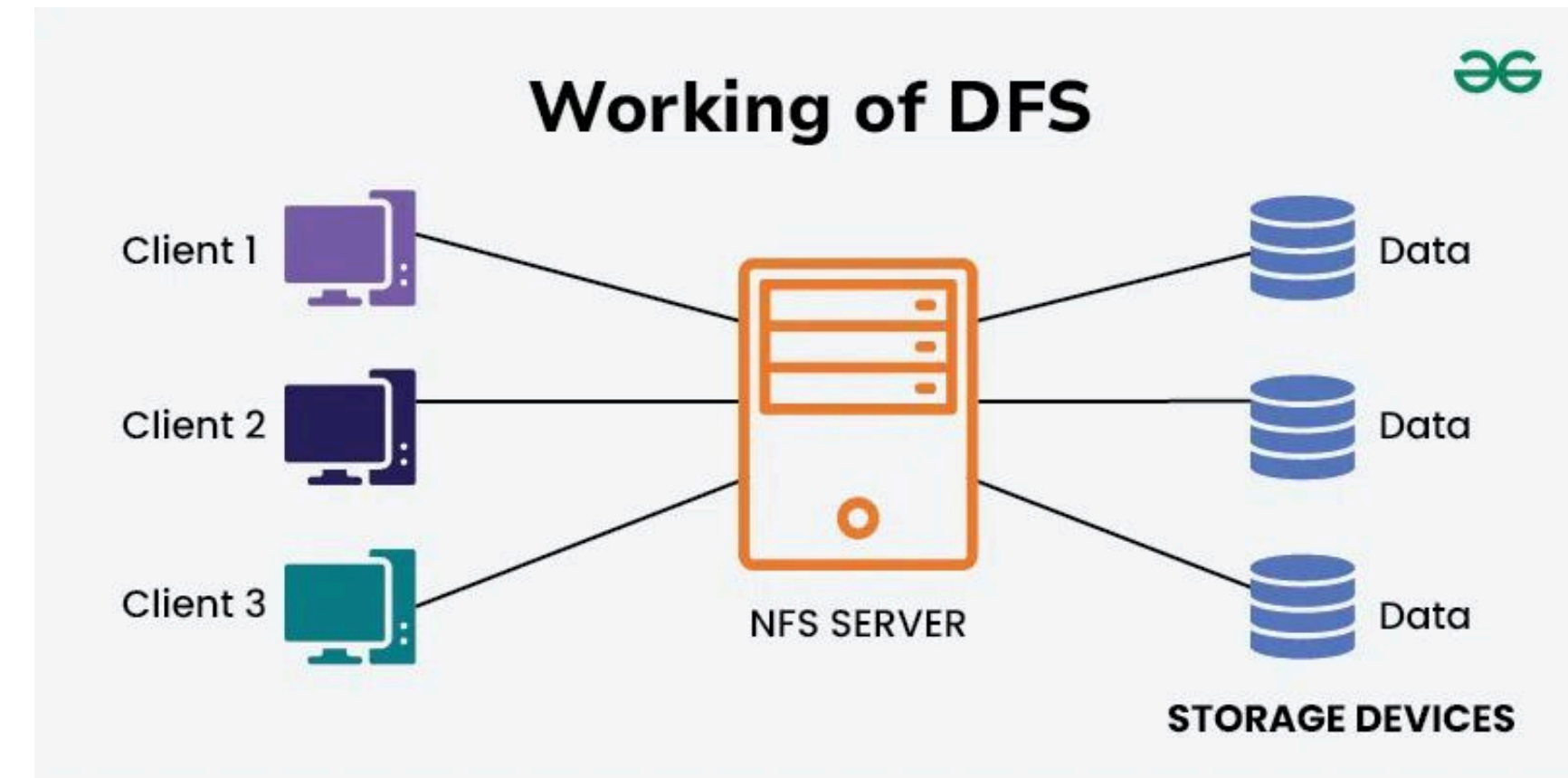
Vous êtes dans la peau d'un data engineer et vous devez mettre en place une architecture médaillon en 1 slide pour répondre à une problématique business de votre choix.

- 1- Chercher votre problématique business à résoudre
- 2- Proposer des sources de données fictives (structurées et non structurées)
- 3- Proposer les traitements que vous pouvez faire entre les différentes couches bronze-> silver et silver->gold

PS: Mettre le plus de détails possibles sur les supports de stockage, les outils de big data à utiliser, les transformations à effectuer et l'impact business cible. N'oubliez pas Google is your best friend.

LES SYSTEMES DE STOCKAGE: DISTRIBUTED FILES SYSTEM

- Système de stockage de fichiers distribués entre plusieurs serveurs et/ou plusieurs lieux géographiques
- Les données sont accessibles en réseau de façon sécurisée
- Les données peuvent être stockées dans un Data center propre à l'entreprise ou sur le cloud

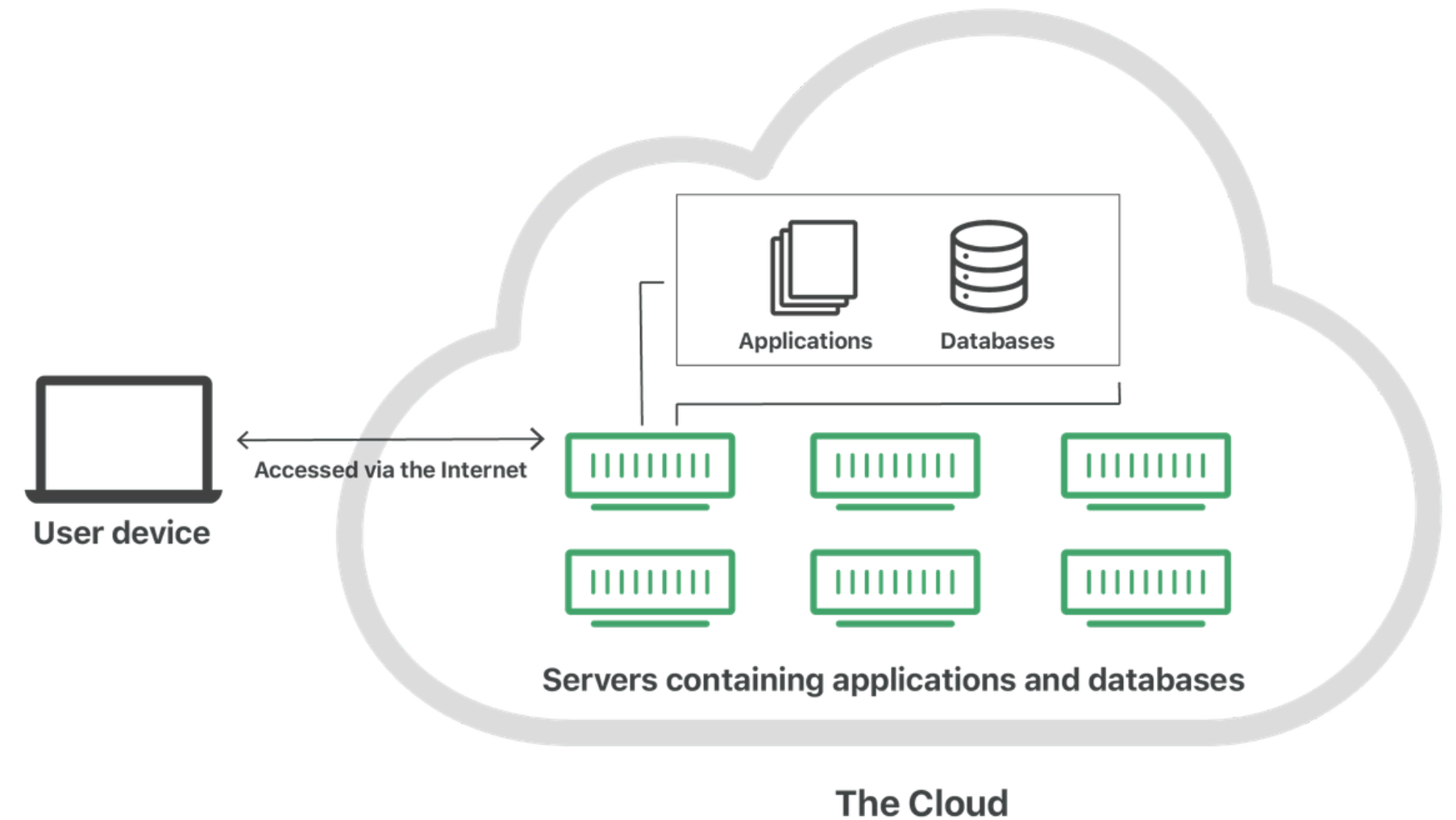


DISTRIBUTED FILES SYSTEM: CARACTERISTIQUES

- Rapidité d'accès: Les données sont plus rapidement accessibles quand la source physique est géographiquement plus proche
- Indépendance au lieu géographique: Pas besoin de savoir où se trouve la donnée pour y accéder.
- Scalabilité: Le DFS s'adapte aisément à l'augmentation de la volumétrie
- Tolérances aux pannes: A travers un système de redondance, la donnée devient indépendante à un cluster ou à un serveur
- Sécurité: Les données sont sécurisées à travers des gestionnaires d'accès

Quelques exemples de DFS: HDFS (Hadoop DFS), Google File System, MapR File System

D'après CNIL (Commission nationale de l'informatique et des libertés), Le cloud computing (en français, « informatique dans les nuages ») fait référence à l'utilisation de la mémoire et des capacités de calcul des ordinateurs et des serveurs répartis dans le monde entier et liés par un réseau.

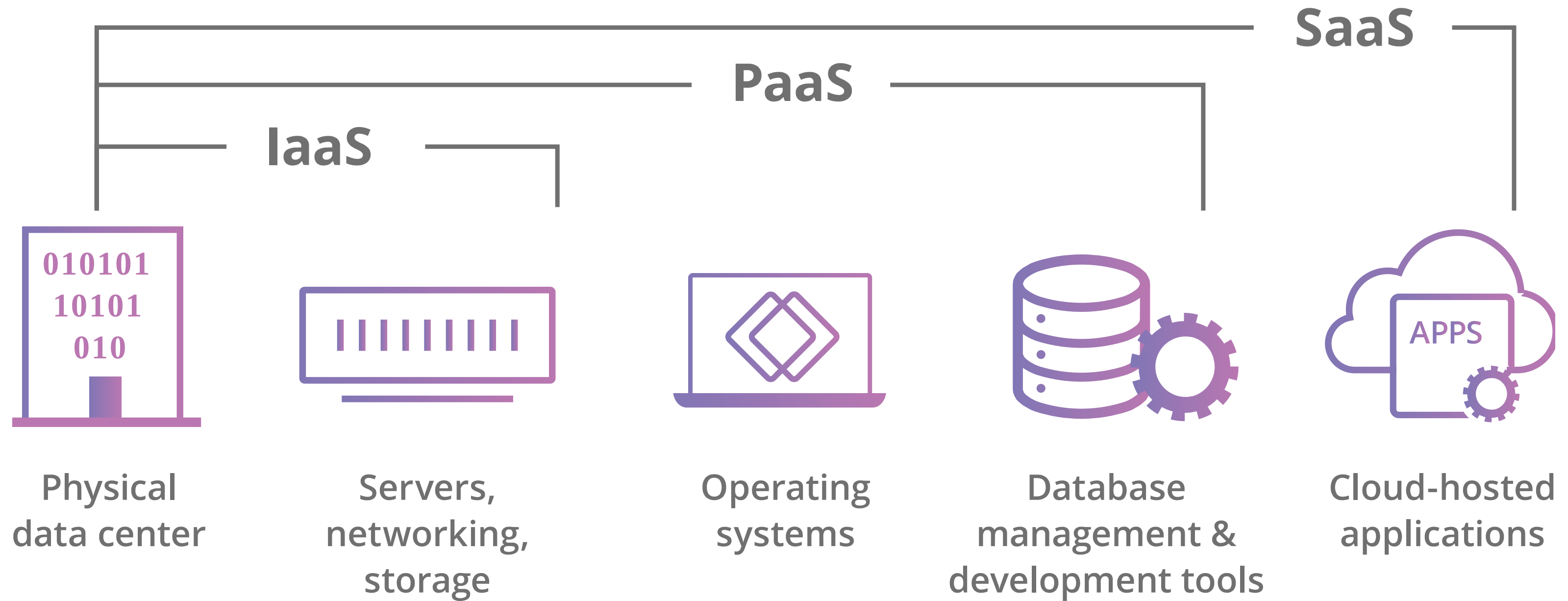


- Flexibilité: Les clients peuvent accéder leurs ressources peu importe leur position géographique.
En outre, ils peuvent adapter leur consommation en fonction de leur besoin
- Efficacité: Les clients peuvent se focaliser sur leurs problématiques fonctionnelles et leur coeur de métier sans se soucier de tout ce qui entoure l'infrastructure matérielle voir logicielle
- Scalabilité: La puissance des ressources s'adapte en fonction du besoin
- Sécurité: Les ressources sont sécurisées et les entreprises peuvent y ajouter leur propre couche de sécurité supplémentaire
- Optimisation des coûts: La facture des ressources s'adapte en fonction du niveau d'utilisation des ressources

- Performance: Le cloud donne accès à de puissantes machines de calcul pour les traitements les plus coûteux
- Green IT: Avec le Cloud, les entreprises peuvent réduire leur empreinte carbone en surveillant leur consommation.

- Sécurité: Le cloud étant un service internet, les ressources peuvent être sujets à des attaques informatiques
- Panne: En cas de problématiques réseaux, les ressources cloud peuvent être inaccessibles (éviter la dépendance forte)
- Confidentialité des données: En fonction de la sensibilité des données, les entreprises ne peuvent pas se permettre de confier leurs données à des tiers
- Coûts imprévus: Une absence d'expertise sur les services peut donner lieu à plusieurs coûts innatendus

CLOUD: LES TYPES DE SERVICE



CLOUD: INFRASTRUCTURE AS A SERVICE

Il consiste à provisionner pour des clients sur le réseau, des infrastructures matérielles (des serveurs/machines). Exemple: Amazon EC2

Avantages pour le client:

- Pas besoin d'assurer la maintenance technique des serveurs ou data centers
- C'est le fournisseur qui est responsable d'assurer la scalabilité physique de son infrastructure et la disponibilité des ressources dans plusieurs pays
- La redondance des données est assurée par le fournisseur
- Plus de temps pour se focaliser sur les problématiques d'architecture fonctionnelle et sur l'applicatif

CLOUD: PLATFORM AS A SERVICE

Il consiste à offrir à ses clients, en plus des services de l'IAAS, un environnement logiciel dédié pour qu'il exécute ses applications. Exemple: Heroku

Avantages pour le client:

- Pas besoin d'assurer la maintenance logicielle des serveurs ou data centers (exemple le système d'exploitation)
- Plus de temps pour se consacrer au Devops , à la conception des applications et à leurs déploiements.

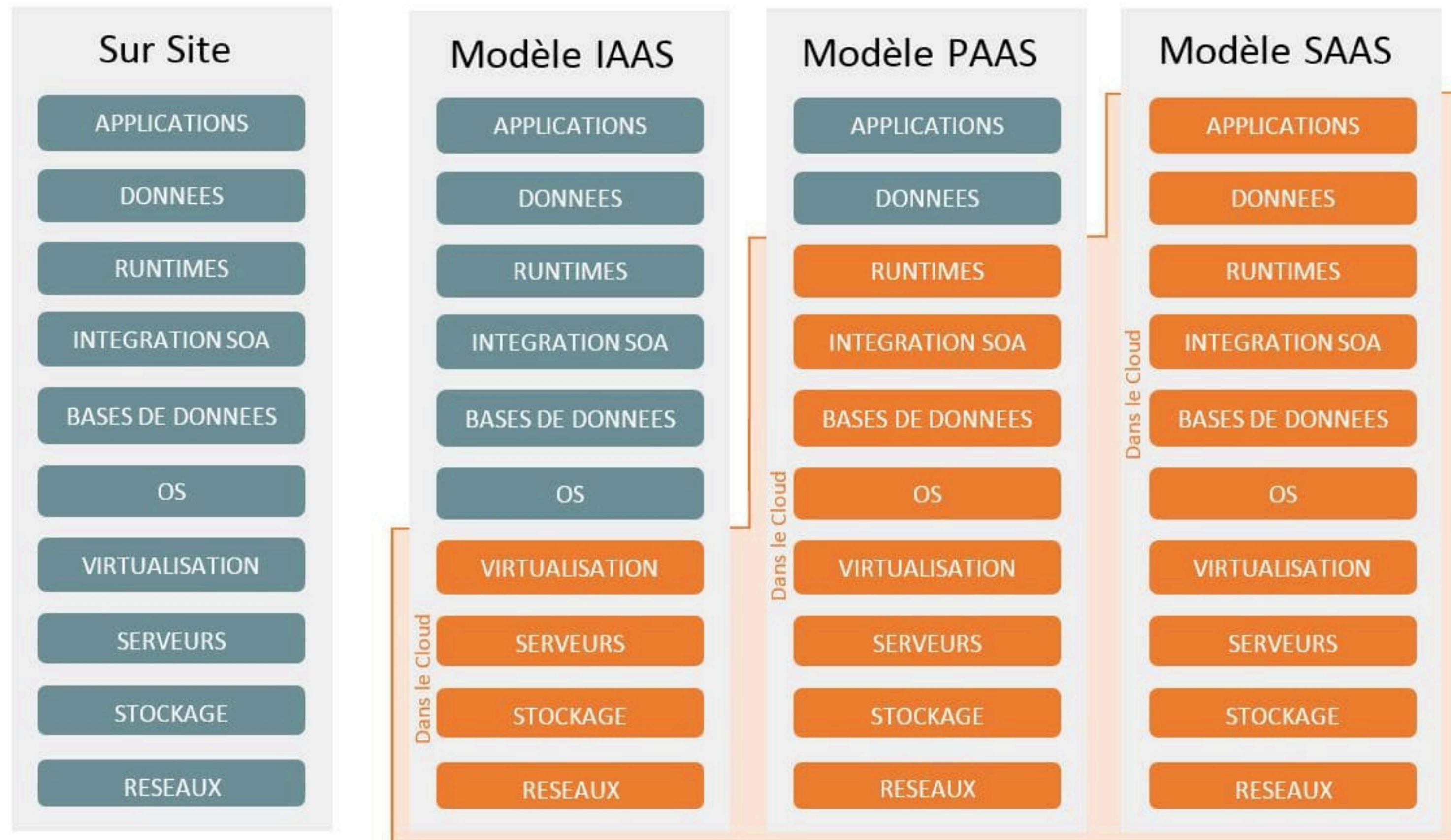
CLOUD: SOFTWARE AS A SERVICE

Il consiste à offrir à ses clients une application clé en main pour personnalisation de façon autonome. Exemple: Shopify

Avantages pour le client (En plus de ceux du PaaS):

- Pas besoin d'assurer la maintenance applicative
- Pas besoin de se soucier des différents bugs
- Le client se focaliser exclusivement sur l'aspect marketing et commercial

CLOUD: RESUME DES INFRASTRUCTURES



TYPE DE CLOUD: CLOUD PRIVE

D'après CloudFare, un cloud privé est un service en cloud qui est exclusivement offert à une organisation. En utilisant un cloud privé, une organisation peut profiter des avantages du cloud computing sans partager les ressources avec d'autres organisations.

Internal Private Cloud



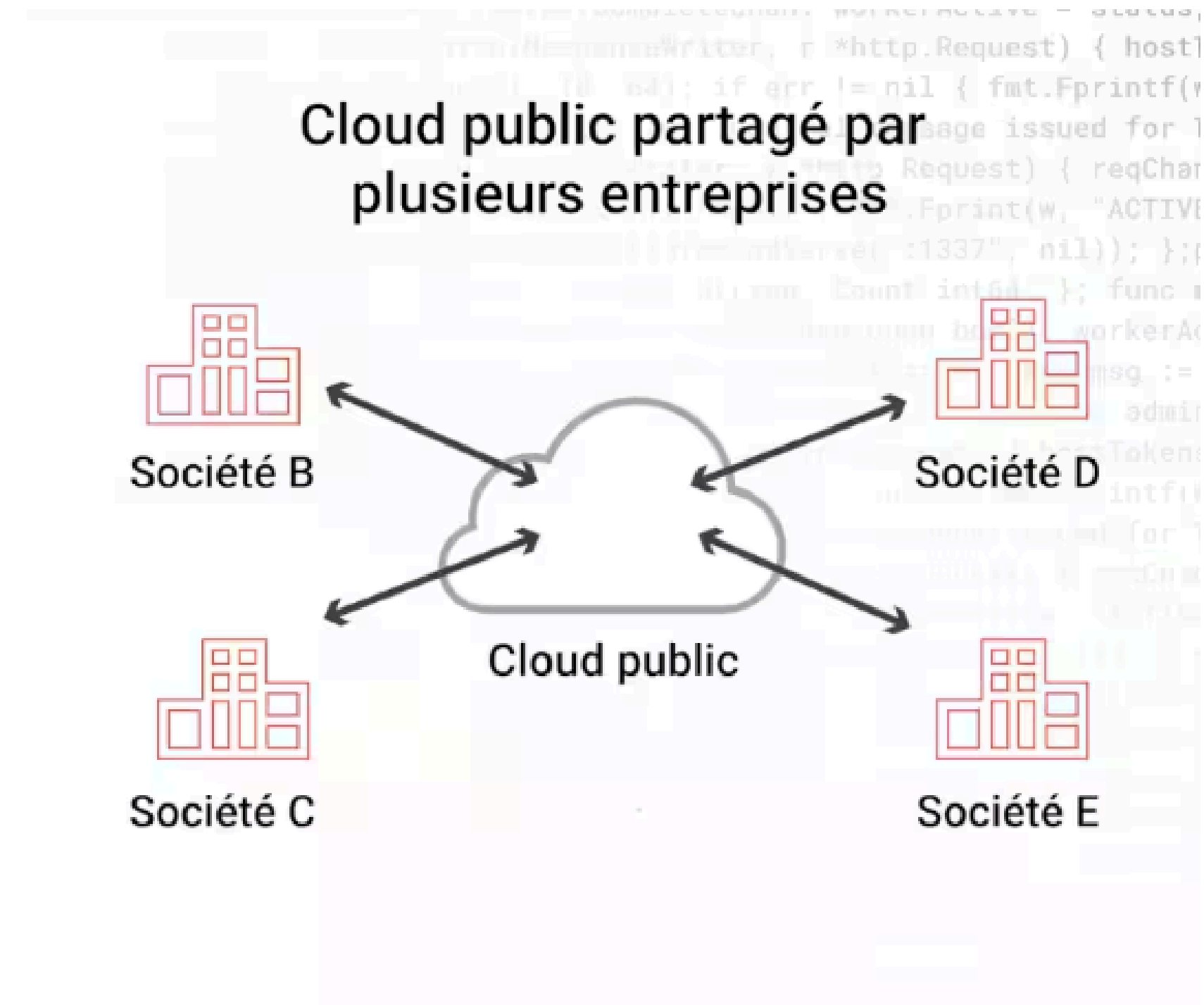
Hosted Private Cloud



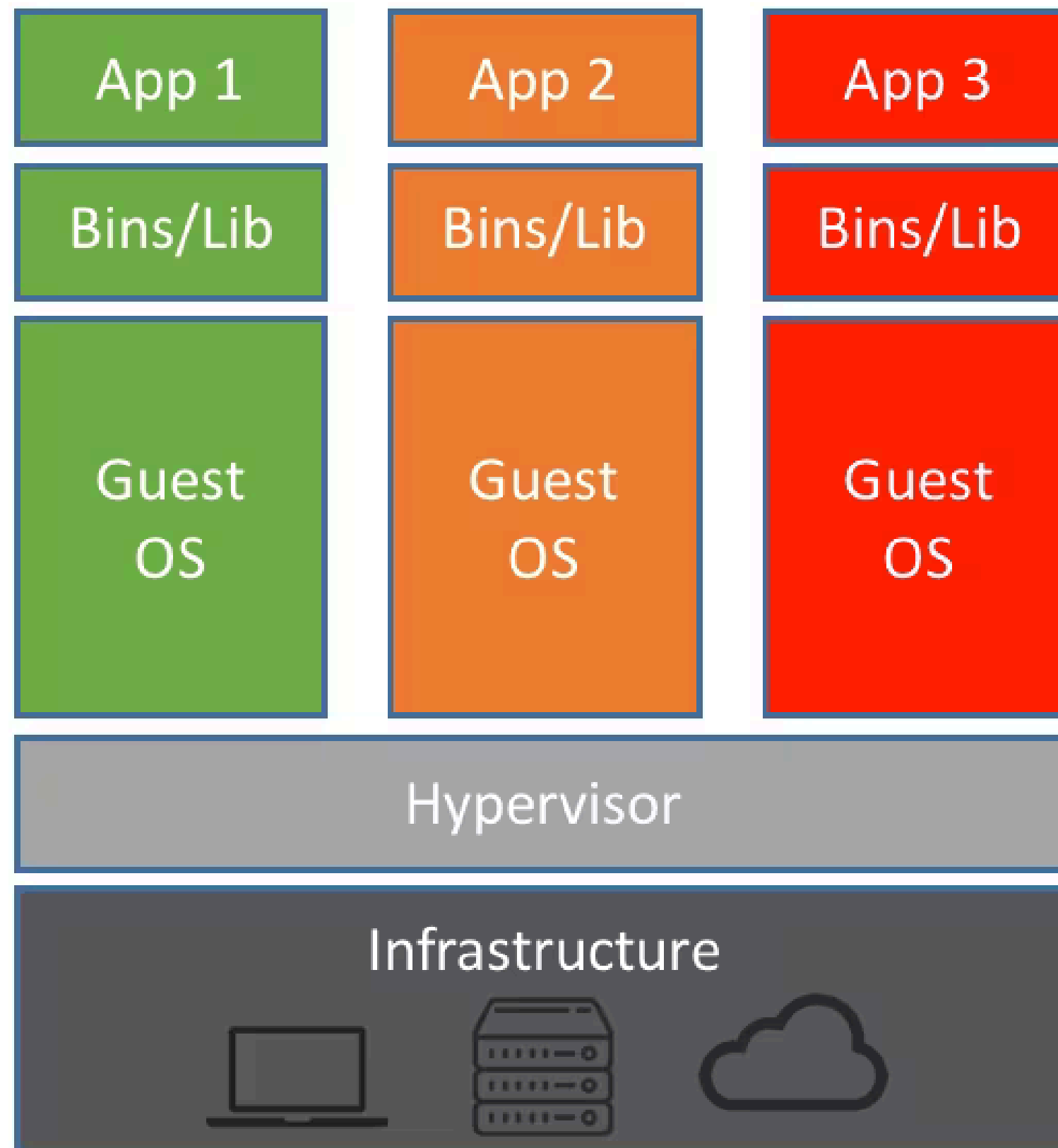
TYPE DE SERVICE CLOUD: CLOUD PUBLIC

Dans un cloud public les ressources sont partagées par plusieurs clients.

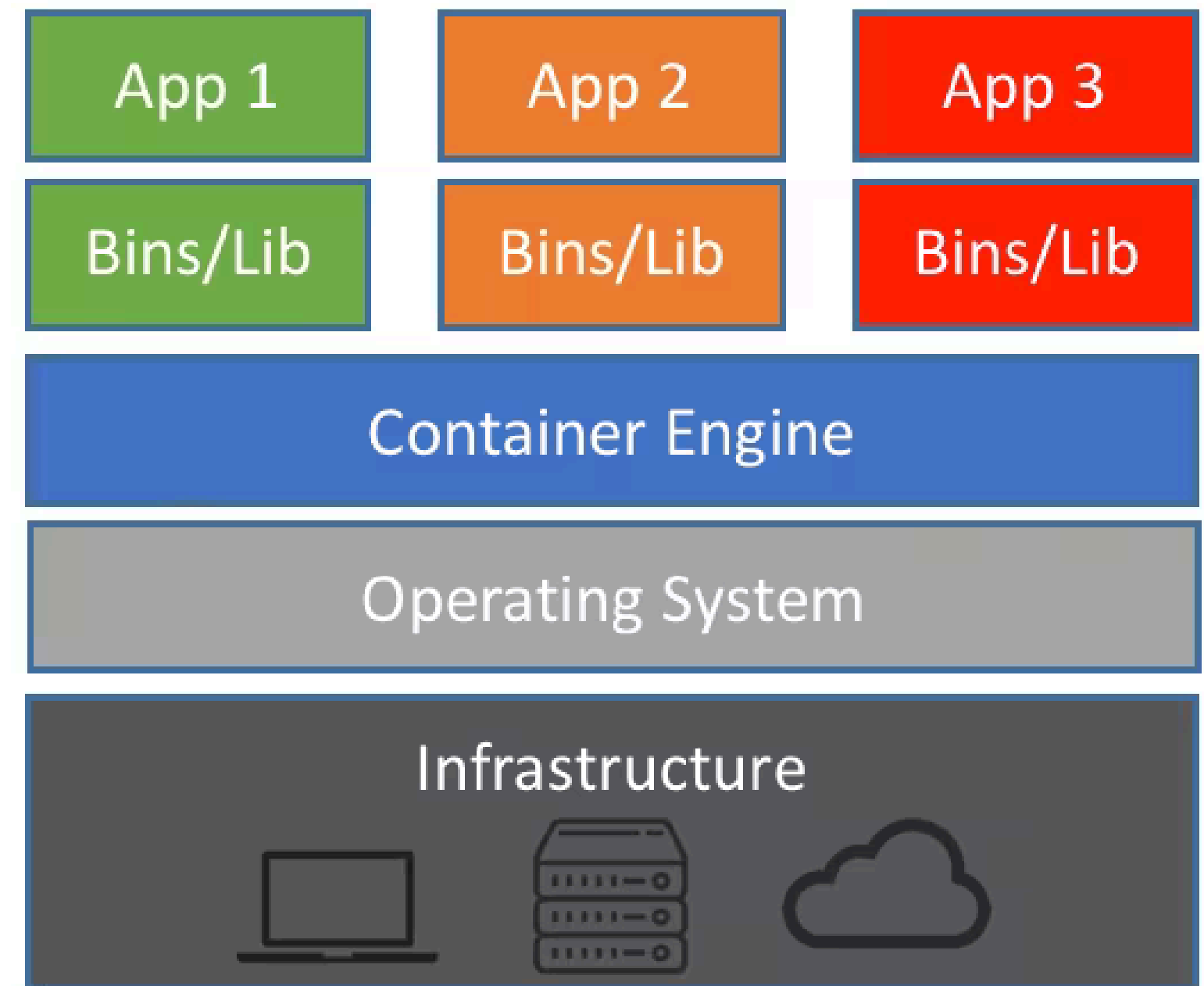
Exemple: un hôtel qui a 100 chambres et chaque chambre peut être payée par n'importe qui à tout de rôle. Les chambres d'un hotel offrent des ressources et on paye en fonction du niveau de qualité des ressources et la durée d'utilisation.



LA VIRTUALISATION



Machine Virtualization



Containers



Questions / Réponses



Evaluation du cours

MODULE 2: METIERS DU BIG DATA

01

Data engineer

02

Data Analyst

03

Data Scientist

C'est un professionnel de la donnée qui a pour objectif d'assurer la collecte, le stockage et la distribution des données en entreprise à très grande échelle.

Actions quotidiennes:

- Collecter les données de plusieurs sources et de plusieurs structures
- Traiter et distribuer les données sous format batch ou en streaming
- Mettre en place des pipelines Big data pour alimenter des datalakes, datawarehouse et datamarts
- Ordonnancer les pipelines Big Data

Rémunération cible: 47 000 euros (Paris)

Great Tools for Data Engineers

Data Platforms



Data Storage



Streaming



Languages



BI Tools



DATA ENGINEER: OUTILS

Storage



BigQuery



Snowflake



Redshift



Clickhouse



Postgres

Orchestration & Transformation



Airflow



DBT



Prefect



Dagster



Databricks

Ingestion (In/Out)



Airbyte



Fivetran



Python



Hightouch



Stitch

Activation



Power BI



Tableau



Metabase



Looker



Whaly

Governance



Monte Carlo



Siffler



Atlan



Castor



Stash

C'est un professionnel de la donnée qui a pour objectif de faire parler les données.

Actions quotidiennes:

- Extraire et nettoyer les données
- Standardiser les données
- Faire du reporting
- Faire des recommandations métiers

Rémunération cible: 45 000 euros (Paris)



DATA ANALYST: OUTILS

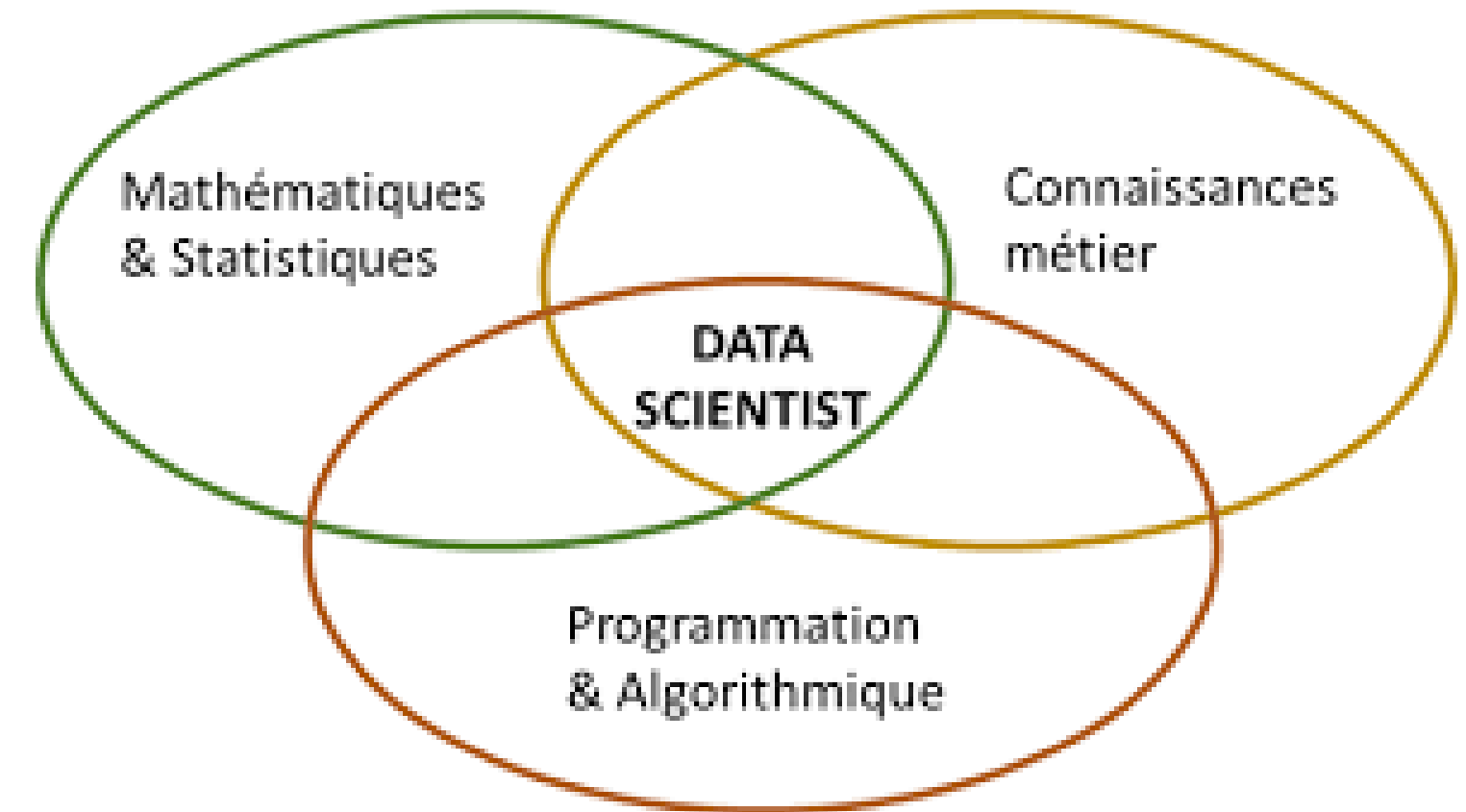


C'est un professionnel de la donnée qui s'appuie sur les données, des formules mathématiques et les algorithmes pour automatiser des résultats, prédire les résultats et trouver des corrélations entre les données.

Actions quotidiennes:

- Extraire et nettoyer les données
- Standardiser et normaliser les données
- Définir des pipelines de preprocessing des données
- Mettre en place des modèles statistiques et de machine learning

Rémunération cible: 49 000 euros (Paris)



DATA SCIENTIST: OUTILS





Questions / Réponses

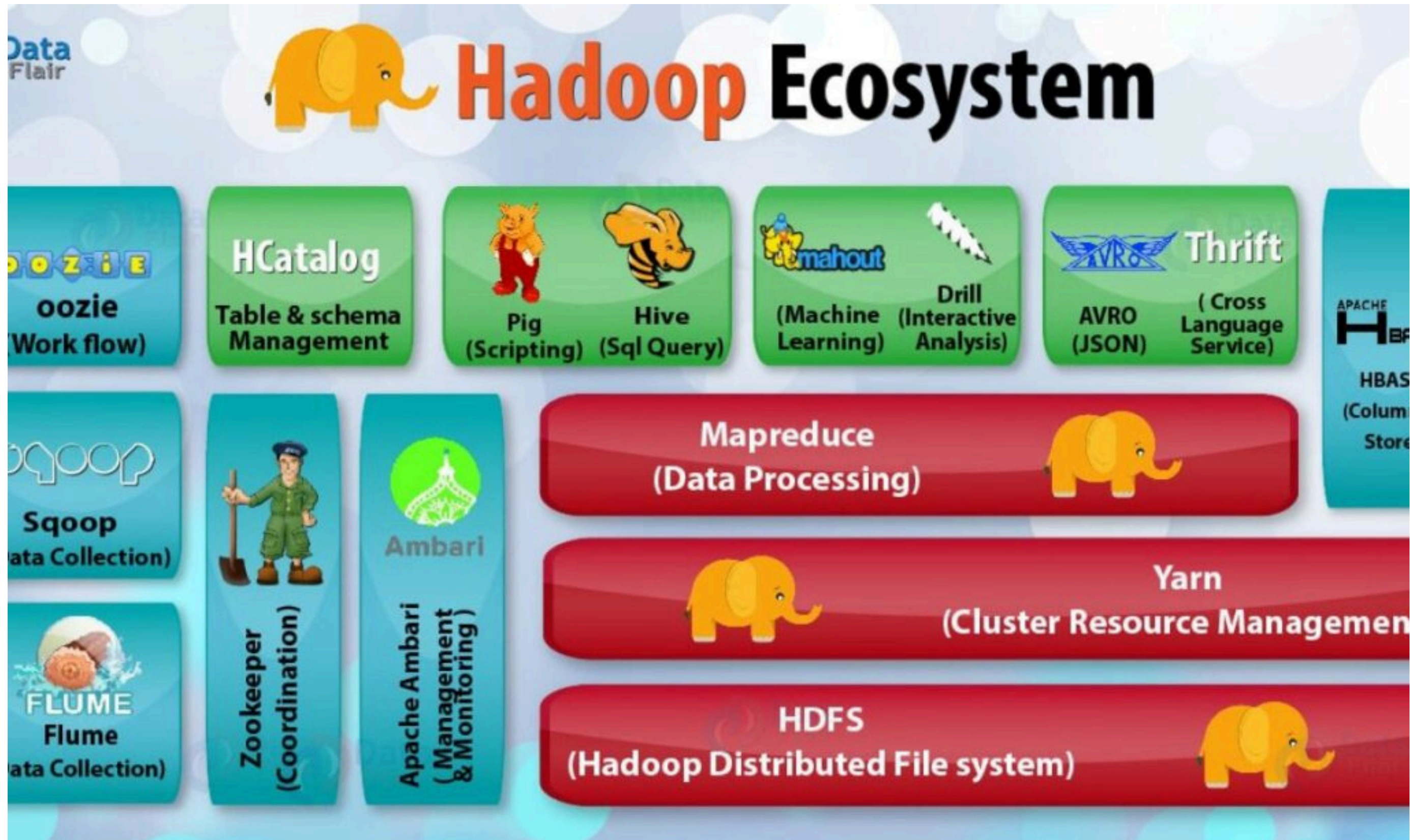


Evaluation du cours

MODULE 3: L'ECOSYSTEME HADOOP 1

- C'est un framework Big Data qui permet de faire des calculs distribués sur un très grand volume de données sur plusieurs machines
- Son écosystème est constitué d'un ensemble de composants permettant de répondre à diverses problématiques notamment le machine learning, l'analyse de données et l'automatisation des traitements
- Il facilite la scalabilité horizontale et verticale des traitements
- Il s'appuie sur une technique de parallélisme de traitements
- Il offre un service de haute disponibilité et une tolérance aux pannes
- Il peut se connecter à plusieurs sources de données et est compatible avec JAVA nativement et avec R, PYTHON et SCALA grâce à des librairies

APACHE HADOOP: DEFINITION



LES CŒURS D'HADOOP (1ERE PARTIE):

HDFS

HDFS (Hadoop File Distributed System) est un système de stockage distribué qui permet à Hadoop de lire des données en entrée et de les stocker en sortie

HDFS repose sur deux serveurs (encore appelés <daemons>):

- Le Namenode: il stocke les informations relatives aux propriétés de distribution des données .

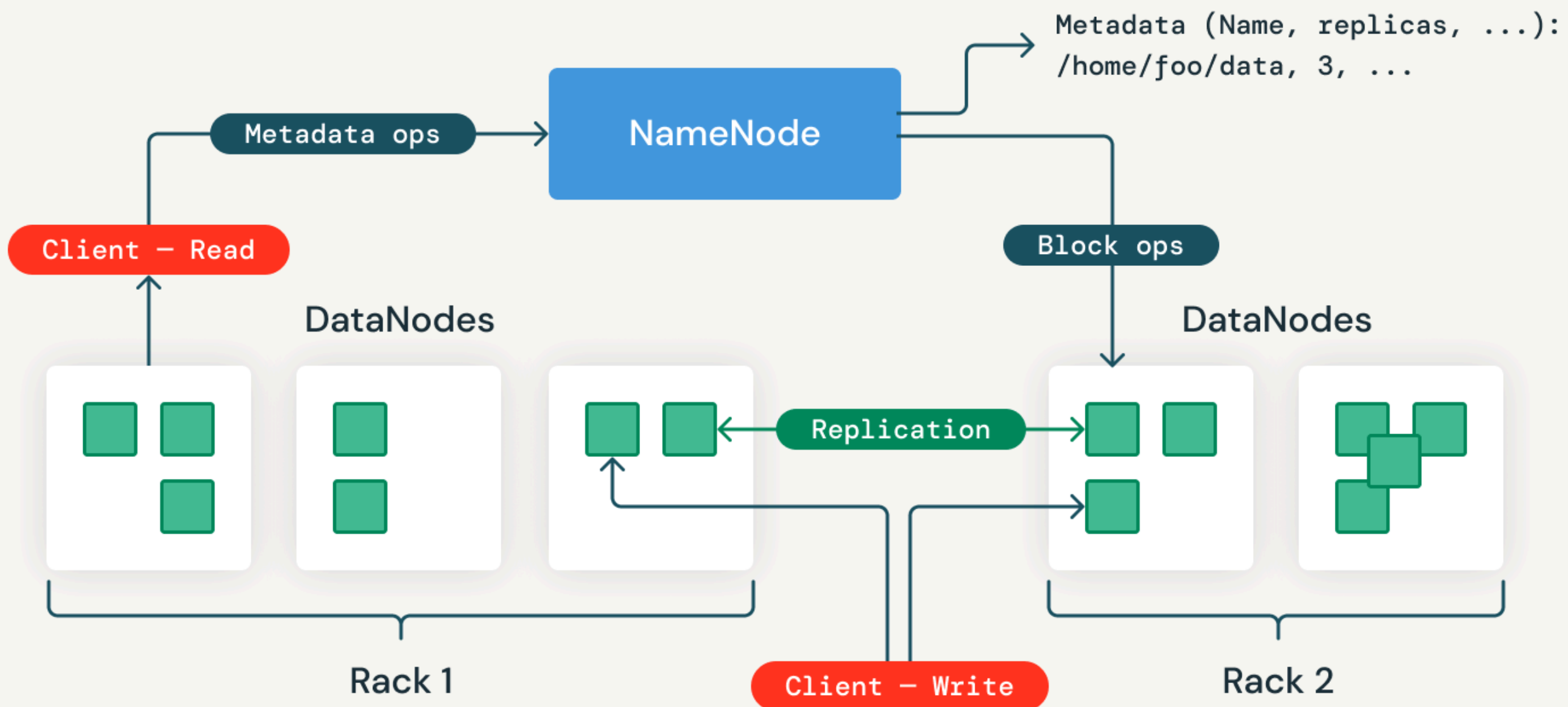
Pour un fichier qui s'appelle <users> dans un répertoire <raw> (découpé en mémoire en 20 blocs) et distribué sur 4 machines , il sait où se trouve chaque bloc. Le namenode est unique.

- Le Datanode: Son rôle est de stocker les blocs de données. Pour une machine, il y a un datanode.

Il est en constante communication avec le Namenode pour recevoir les données et envoyer à ce dernier les informations relatives à leur stockage

LES CŒURS D'HADOOP (1ERE PARTIE): HDFS

HDFS Architecture



LES CŒURS D'HADOOP (2E PARTIE): MAP REDUCE

Problématique: Compter le nombre de personnes dans un hôtel qui a 100 étages

Approche classique: Une personne part dans chaque étage de l'hôtel et compte dans chaque chambre le nombre de nombres.

On va supposer qu'elle prend 10 min par étage. Donc pour 100 étages on a 1000 minutes



LES CŒURS D'HADOOP (2E PARTIE): MAP REDUCE

Problématique: Compter le nombre de personnes dans un hôtel qui a 100 étages

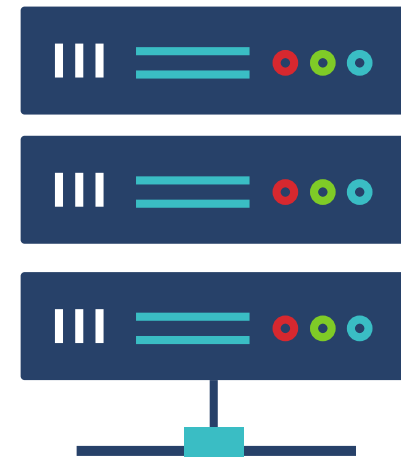
Approche calcul distribué: On a 100 personnes qui partent dans chaque étage de l'hôtel et compte dans chaque chambre le nombre de nombres.

On va supposer qu'une personne prend 10 min par étage. Donc pour 100 étages on a 10min (+ temps de collecte des données de chacun et temps de partage des étages). *

Somme toute ca prendra maximum 20min.



LES CŒURS D'HADOOP (2E PARTIE): MAP REDUCE



BLOCS DE 256MO



BLOCS DE 256MO



BLOCS DE 256MO



BLOCS DE 256MO

Cluster de 20 000 serveurs avec ces blocs de
données => 5To de données traitées

LES CŒURS D'HADOOP (2E PARTIE):

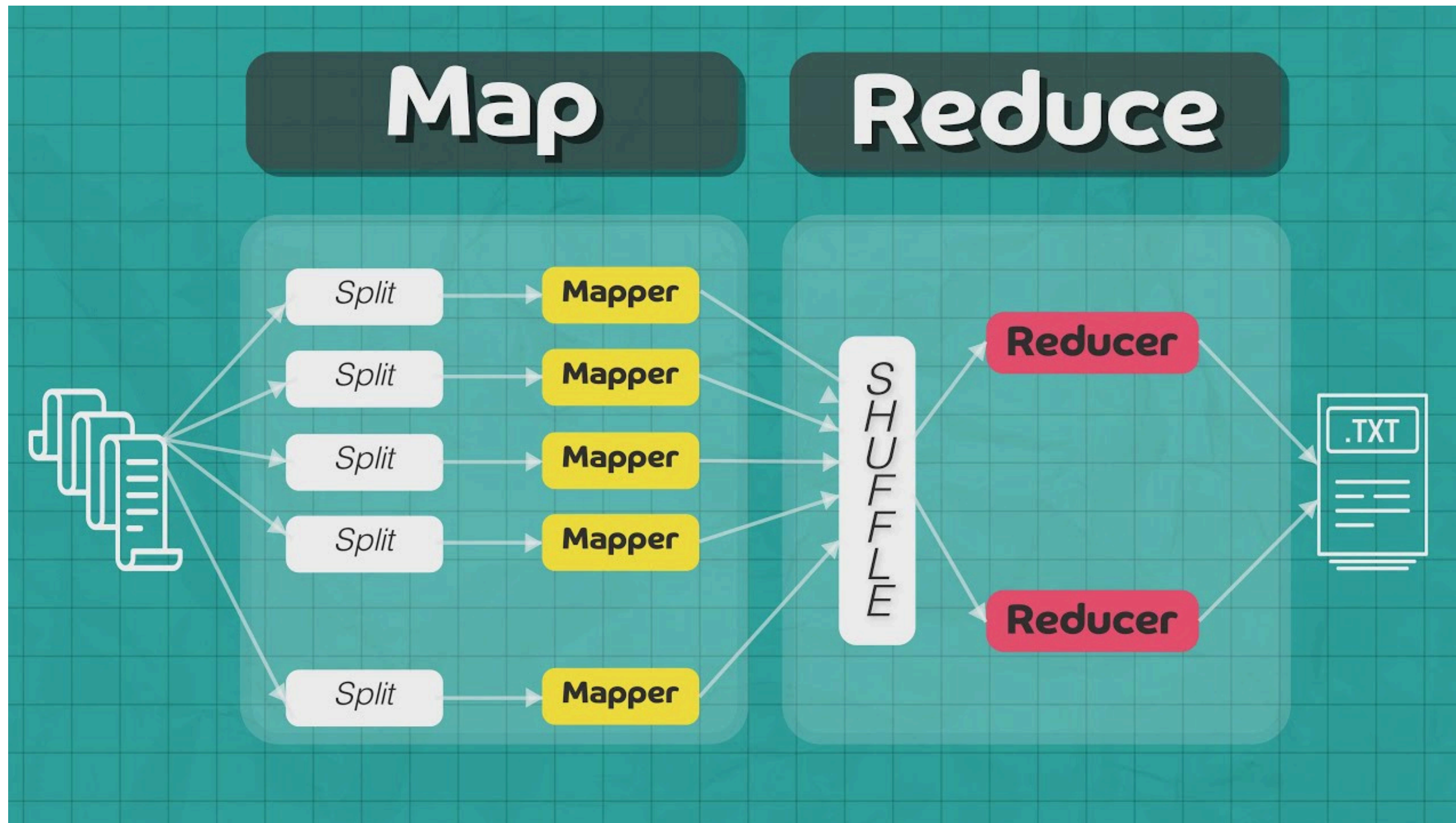
MAP REDUCE

C'est un modèle de programmation qui s'appuie sur HDFS pour faire des traitements avec du calcul distribué dans l'environnement HADOOP.

Il s'appuie sur 3 opérations:

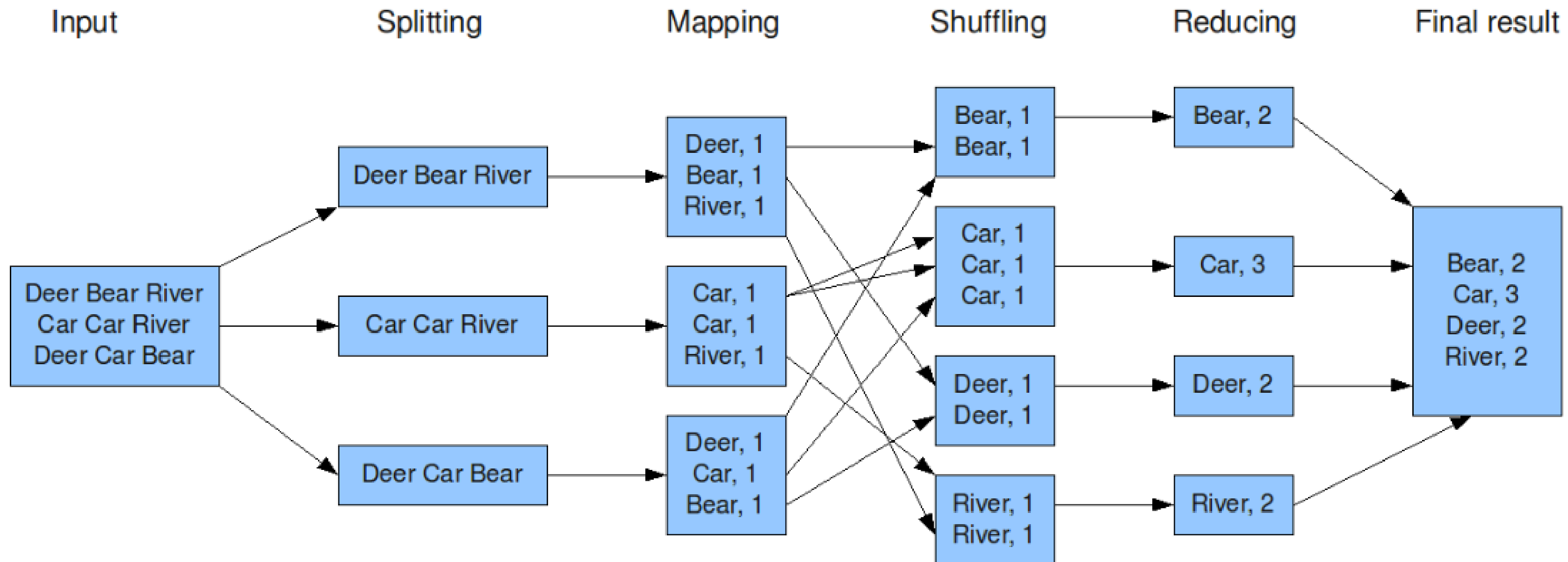
- **MAP:** Cette opération consiste à transformer les données d'entrée en plusieurs blocs avec un système de clé/valeur. Pour un bloc précis, on associe une clé. Ensuite sur chaque bloc on applique la fonction associée au mapping et on génère au sein de chaque bloc, un autre système de clé/valeur.
- **REDUCE:** Après l'étape de mapping, les nouvelles paires de clé/valeur sont redistribuées entre les blocs et regroupées par clé. Après ce regroupement, on combine les valeurs issues de la même clé et on écrit les résultats sur HDFS

LES CŒURS D'HADOOP (2E PARTIE): MAP REDUCE



LES CŒURS D'HADOOP (2E PARTIE): MAP REDUCE USE CASE

The overall MapReduce word count process



LES CŒURS D'HADOOP (3EME PARTIE):

YARN

YARN (Yet Another Resource Negotiator) est le gestionnaire de ressources d'Hadoop. . Il fournit une base sûre et stable pour la gestion opérationnelle et le partage des ressources du système, pour une efficacité et une flexibilité maximale.

Son rôle:

- Il alloue des ressources pour chaque traitement
- Il surveille les traitements exécutés dans les différentes machines
- Il coordonne l'exécution des applications de façon distribuée

LES CŒURS D'HADOOP (3EME PARTIE):

YARN

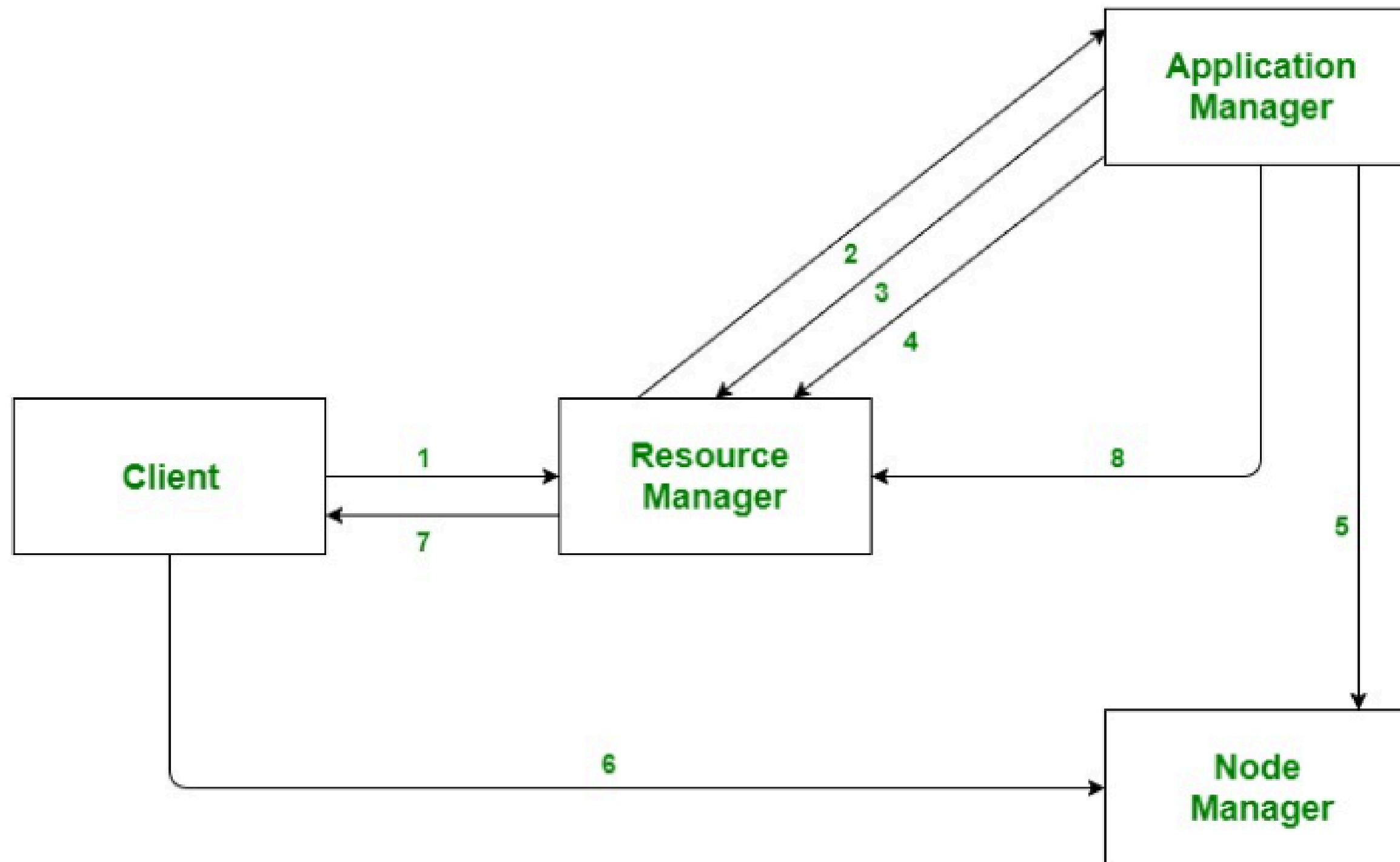
Yarn est composé de 6 composants:

- Ressource Manager: C'est lui qui reçoit les instructions d'exécution des jobs (job = programme à exécuter) à l'entrée, alloue les ressources et programme leur exécution
- Scheduler: C'est un composant du RM. Il programme les exécutions des jobs en fonction des ressources disponibles et de la demande des applications
- ApplicationManager: C'est le composant du RM qui reçoit les jobs et négocie la création des conteneurs dans un noeud chez le ResourceManager
- ApplicationMaster: Contenu dans chaque noeud et associé à une application unique, il supervise l'exécution du job de façon distribuée. Il produit également les logs applicatifs associés

LES CŒURS D'HADOOP (3EME PARTIE): YARN

- NodeManager: Il surveille les exécutions dans chaque noeud (une machine en réseau dans le cluster) où il se trouve (1 noeud = 1 NodeManager) et rapporte les données au Resource Manager. Somme toute, il assure le monitoring d'infrastructure du Node (RAM, CPU, Statut de santé des nodes, etc...)
- Container: Ils sont contrôlés par les NodeManagers. Ils contiennent les ressources physiques nécessaires pour l'exécution des jobs (RAM, CPU, espace disque) sur un node.

PROCESSUS DE YARN



- 1- Un job (un traitement) est soumis à l'écosystème Hadoop. Le gestionnaire de ressources d'Hadoop (Yarn) s'active et envoie le besoin au Ressource Manager
- 2- Le RM crée un conteneur pour démarrer l'Application Manager
- 3- L'AM s'enregistre chez le RM
- 4- L'AM négocie les ressources nécessaires pour l'exécution du job (les conteneurs nécessaires)
- 5- L'AM demande au NodeManager de créer les conteneurs dans lesquels seront exécutés les traitements
- 6- Les conteneurs exécutent le job de façon distribuée
- 7- Le client peut demander les métriques de suivi des traitements grace au RM
- 8- Le gestionnaire d'applications se désinscrit du RM et la mémoire est libérée pour d'autres jobs

Challenge...



> 950
Million
Users

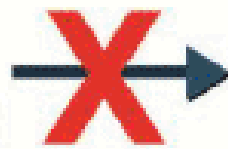
> 500
Terabytes
per day

> 70k
queries
per day

> 300
million
photos per
day

Traditional RDBMS... **X**

Solution...



Hard to
program

Users
know
SQL well



Tables can be
partitioned and
bucketed

Schema
flexibility and
evolution

Easy to plug-in
custom
mapper/
reducer code

JDBC/ODBC
drivers are
available

HIVE tables
can be defined
directly on
HDFS

Extensible:
Types, Formats,
Functions,
Scripts

C'est un datawarehouse qui permet de faire de l'analyse de données en s'appuyant sur HiveQL (qui est semblable à SQL).

Les caractéristiques de HIVE:

- Il organise les données en tableau dans le répertoire HDFS. Les tableaux sont similaires à une base de données relationnelle.
- Les données ont chacun un numéro et chaque tableau correspond à un répertoire HDFS
- Les tableaux dans Apache HIVE sont partitionnés
- Il traduit les requêtes en HiveQL en un ensemble de jobs MapReduce (Spark ou autre exécutateur distribué).

L'écosystème Hadoop est constitué d'autres composants parmi lesquels:

- Ambari: Outil qui permet la gestion simplifiée, la création et l'audit des clusters Hadoop.
- Pig: Plateforme qui permet d'exécuter des programmes MapReduce en s'appuyant sur un langage de programmation simplifié appelé Pig Latin
- Oozie: Composant permettant la mise en place de workflows Big Data. En d'autres termes on peut ordonnancer et planifier des jobs
- Mahout: Il permet l'entraînement des modèles de Machine Learning de façon distribuée grâce à MapReduce
- Hbase: Base de données NoSQL de type colonne s'appuyant sur HDFS



Questions / Réponses

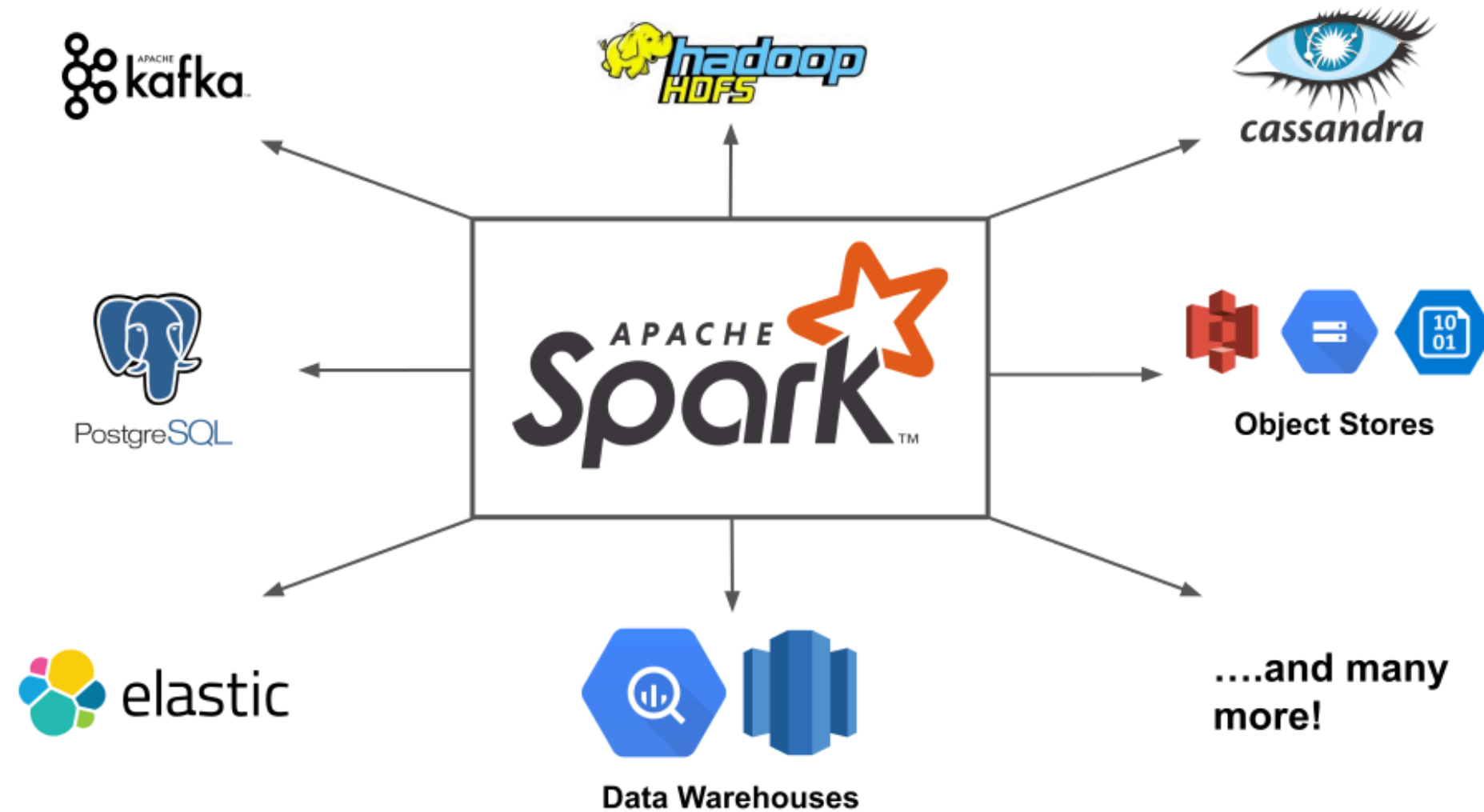


Evaluation du cours

MODULE 4: L'ECOSYSTEME SPARK

- Traitement des fichiers pas assez optimal à cause des opérations de lecture/écriture sur disque et des traitements séquentiels de Map/Reduce
- Incapacité à effectuer des calculs en temps réel
- Incapacité à appliquer de façon itérative un traitement sur les même données
- Difficultés d'utilisation aussi bien pour la programmation des traitements que sur le debugging en cas d'échec.

LES ALTERNATIVES A HADOOP MAPREDUCE



Flink

SPARK: INTRODUCTION

- C'est un framework Big Data écrit en Scala, qui permet de faire des calculs distribués en mémoire sur plusieurs machines sur un très grand volume de données
- Il reprend les mêmes avantages qu'Hadoop et corrige une grande partie de ces limites
- Il se caractérise par un traitement In-Memory des données
- Il n'a qu'une phase lecture en entrée de HDFS et une phase écriture à la fin de l'opération
- Il manipule 2 types d'objets qui facilitent les traitements distribués : Les Dataframes et les RDD (Resilient Distributed Dataset). Ces objets sont stockés en cache en mémoire
- Il peut se connecter à plusieurs sources de données et est compatible nativement avec JAVA, R, PYTHON et SCALA
- Il permet de faire des traitements en batch, micro-batch et en streaming



APACHE SPARK ECOSYSTEM

Spark SQL

**Spark
Streaming**
(Streaming)

MLlib
(Machine
learning)

GraphX
(Graph
Computation)

SparkR
(R on spark)

Apache Spark Core API

R

SQL

Python

Scala

Java



Les Resilient Distributed Dataset sont des collections immuables d'objets qui effectuent des calculs sur plusieurs nœuds d'un cluster. Les RDDs constituent la structure de données fondamentale de SPARK.

L'acronyme RDD:

- Resilient: Les RDDs offrent une tolérance à la panne grâce à leur système de Data Lineage avec des DAGs (prochain chapitre).
- Distributed: Les Datasets sont sur plusieurs nœuds d'un cluster
- Datasets: Ce sont des enregistrements de données avec lesquels nous travaillons

Les RDD sont automatiquement partitionnés sur plusieurs serveurs et peuvent être traités sur plusieurs nœuds d'un cluster

- Les RDDs sont stockés en mémoire RAM pour des traitements plus rapides
- Les opérations sont dit “paresseuses” ce qui veut dire qu’elles ne sont exécutées que si elles sont nécessaires (par contre elles sont chargées en mémoire)
- Il y a la tolérance aux pannes
- Les objets sont immuables: quand ils sont écrits en mémoire on ne peut plus les modifier



- Les données sont distribuées sur plusieurs noeuds pour faciliter le parallélisme des traitements
- Les données peuvent être persistées aussi bien en mémoire que sur disque
- Les RDDs peuvent être partitionnés rendant très flexible la distribution des données sur les nodes et sur le parallélisme
- Les RDD prennent charge 2 types d'opérations: Les transformations et les Actions



Il existe 2 types d'opérations sur les RDDs.

- Les transformations: Ce sont des fonctions qui ont pour objectif de créer de nouveaux RDDs à partir d'un RDD existant ou en chargeant des données issues d'une source.

Exemple: Map(), Filter() ou Sample()

- Les actions: Ce sont des fonctions qui ont pour objectif de créer le résultat issus de plusieurs transformations. Il produit une donnée qui n'est pas type RDD et qui est stocké en mémoire dans le Driver. C'est également à travers ces fonctions qu'on peut exporter les résultats vers n'importe quel support de stockage. Exemple: First(), take() et count()

SPARK: RDD, DATAFRAME ET DATASETS

Les RDD: Ils ont utilisés pour les données non structurées ou sans schéma prédéfini nécessitant des calculs basiques (map, filter, etc..) et les calculs de groupements..

Commande associée (Python): `sc.parallelize` (pour les collections) et `sc.textFile` pour charger un fichier texte..

Les Dataframes: Ils permettent de manipuler la donnée avec des opérations de type SQL. C'est très pratique pour les traitements qui nécessitent pas une logique métier complexe. C'est l'idéal pour manipuler les données structurées.

Commande associée (Python): `spark.read.csv` pour charger les csv

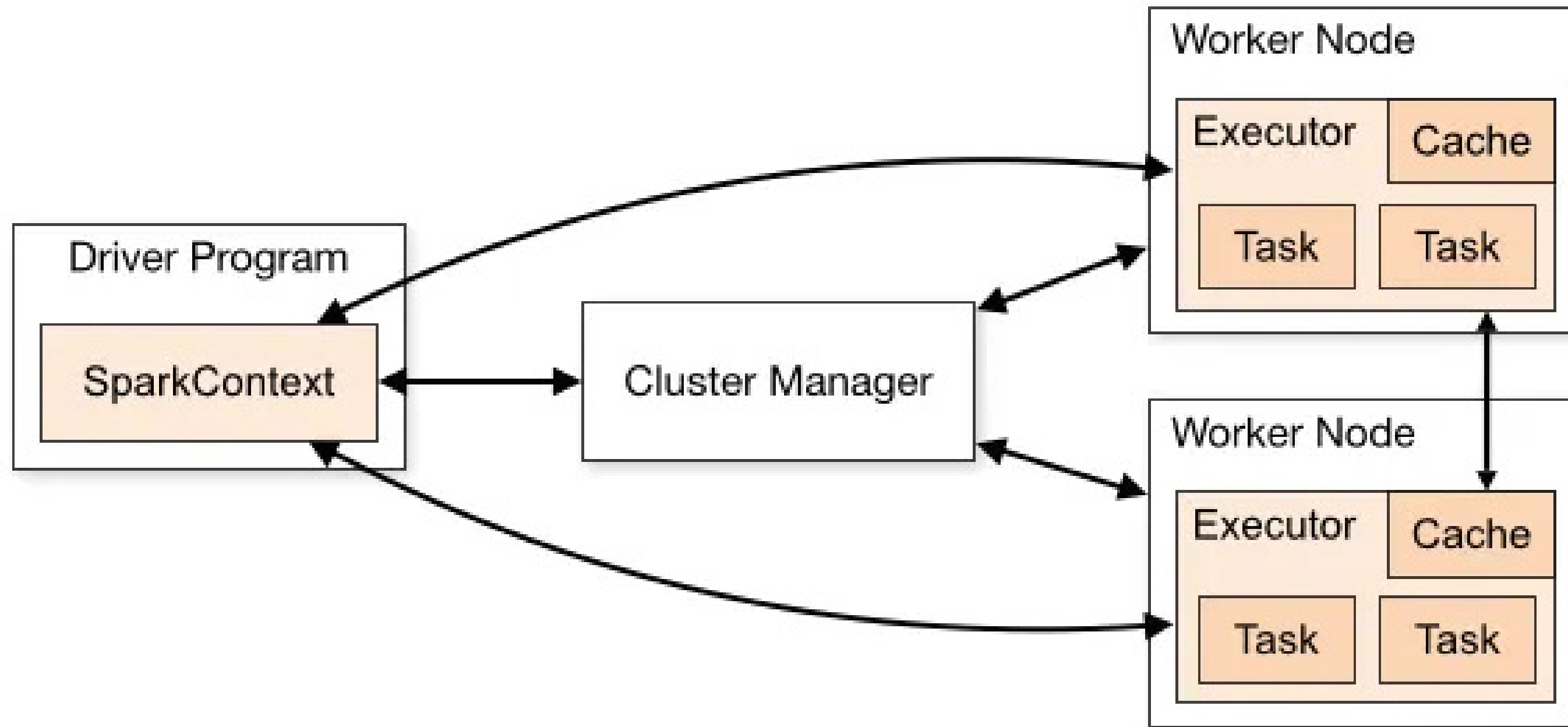
Les Datasets: Ils permettent d'avoir le meilleur des Dataframes et des RDDs. ils sont recommandés pour manipuler des données fortement typés et bénéficier des avantages associés lors de la compilation.

Ils combinent les avantages des 2 précédentes structures de données. De façon sommaire les Dataframes sont des datasets non typés.

Il existe des commandes pour naviguer d'une structure à une autre.

Attention: ils ne sont disponibles qu'en Java et en Scala.

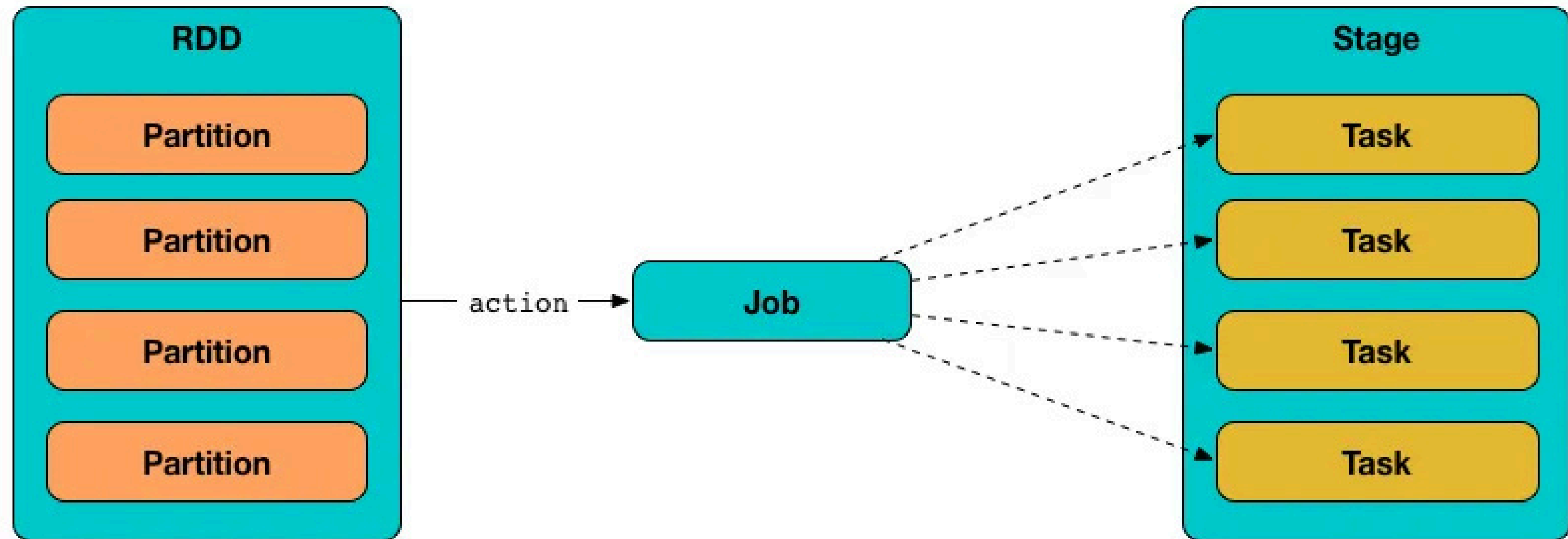
L'ARCHITECTURE SPARK



- **SPARK DRIVER :** C'est le programme central qui se charge du bon fonctionnement d'une application SPARK. Il crée un connecteur qu'on appelle le SparkContext qui a pour objectif d'établir le lien avec les clusters Spark et de monitorer les traitements sur les données.
- **SPARK CONTEXT:** Il établit le lien entre le SPARK DRIVER et le CLUSTER MANAGER. Il coordonne l'exécution des tâches et il permet de créer les variables nécessaires pour manipuler les données notamment les RDDs.
- **SPARK EXECUTOR:** C'est un programme qui exécute les différentes tâches nécessaires pour un traitement SPARK. Il assure les opérations de lecture/écriture sur disque. Il tourne sur un WORKER Node.
- **CLUSTER MANAGER:** Il est responsable de l'allocation des ressources et la gestion du cluster.

- **STANDALONE CLUSTER:** C'est le cluster manager natif de SPARK. Il est utilisé pour les clusters de petites et de moyennes tailles. C'est l'idéal pour les projets en développement ou pour les tests car il contient les fonctions basiques pour manager un cluster.
- **YARN:** C'est le gestionnaire de ressources de l'écosystème HADOOP. Il permet une grande scalabilité contrairement au mode Standalone. Toutefois il nécessite une configuration complexe pour s'interfacer avec SPARK.
- **MEZOS:** Il offre les mêmes avantages que YARN à l'exception de la forte compatibilité avec HADOOP. Il est préconisé lorsqu'on veut une forte isolation entre les clusters (Sécurité).
- **KUBERNETES:** C'est un orchestrateur de conteneurs. C'est l'idéal lorsque les applications sont déployées dans des conteneurs.

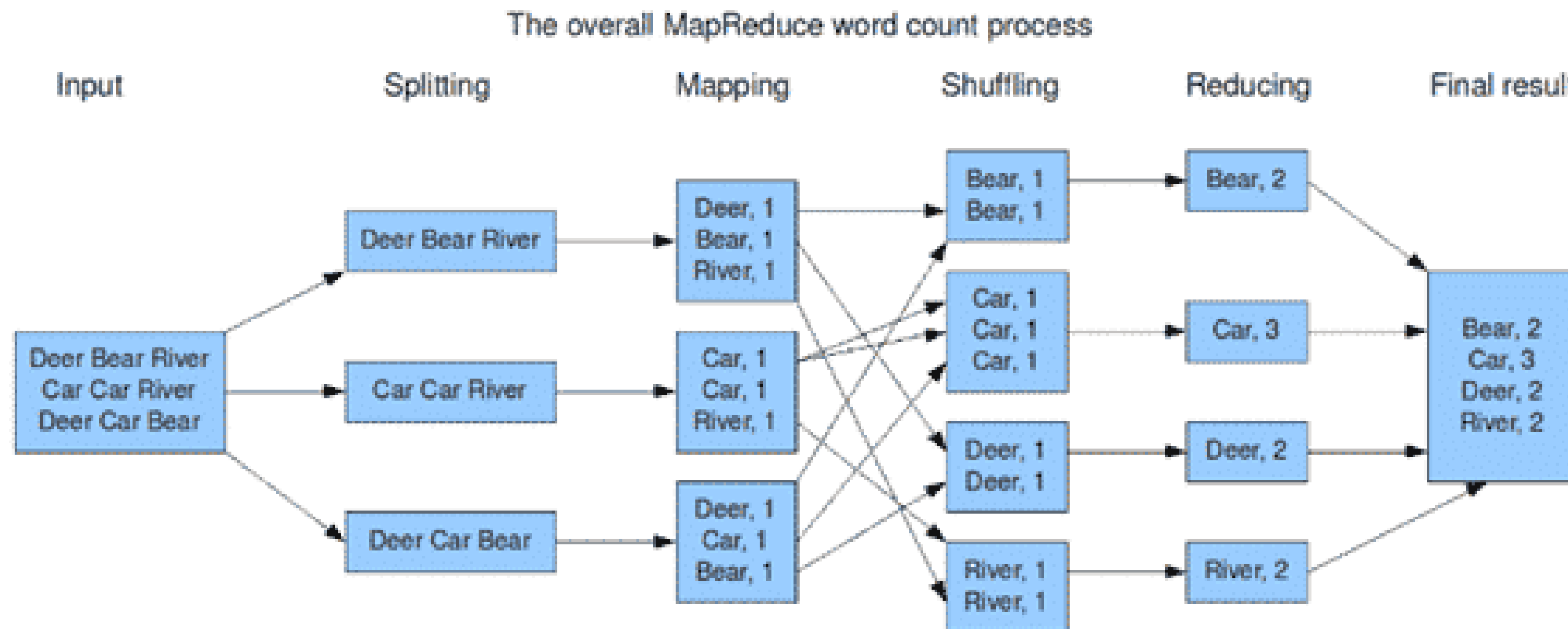
FONCTIONNEMENT INTERNE D'UN RDD: CONCEPTS CLES



FONCTIONNEMENT INTERNE D'UN RDD:

CONCEPTS CLES

Le shuffling est une opération qui consiste à déplacer des données d'une machine à une autre en créant des partitions qui ont la même clé



Les Jobs: Un job est un ensemble de transformations appliquées sur une donnée. Chaque fois qu'une action est appelée sur un RDD, un job est créé. Un programme SPARK peut contenir plusieurs actions et ainsi pourrait donner lieu à plusieurs jobs en cours. Un job se subdivise en plusieurs Stages.

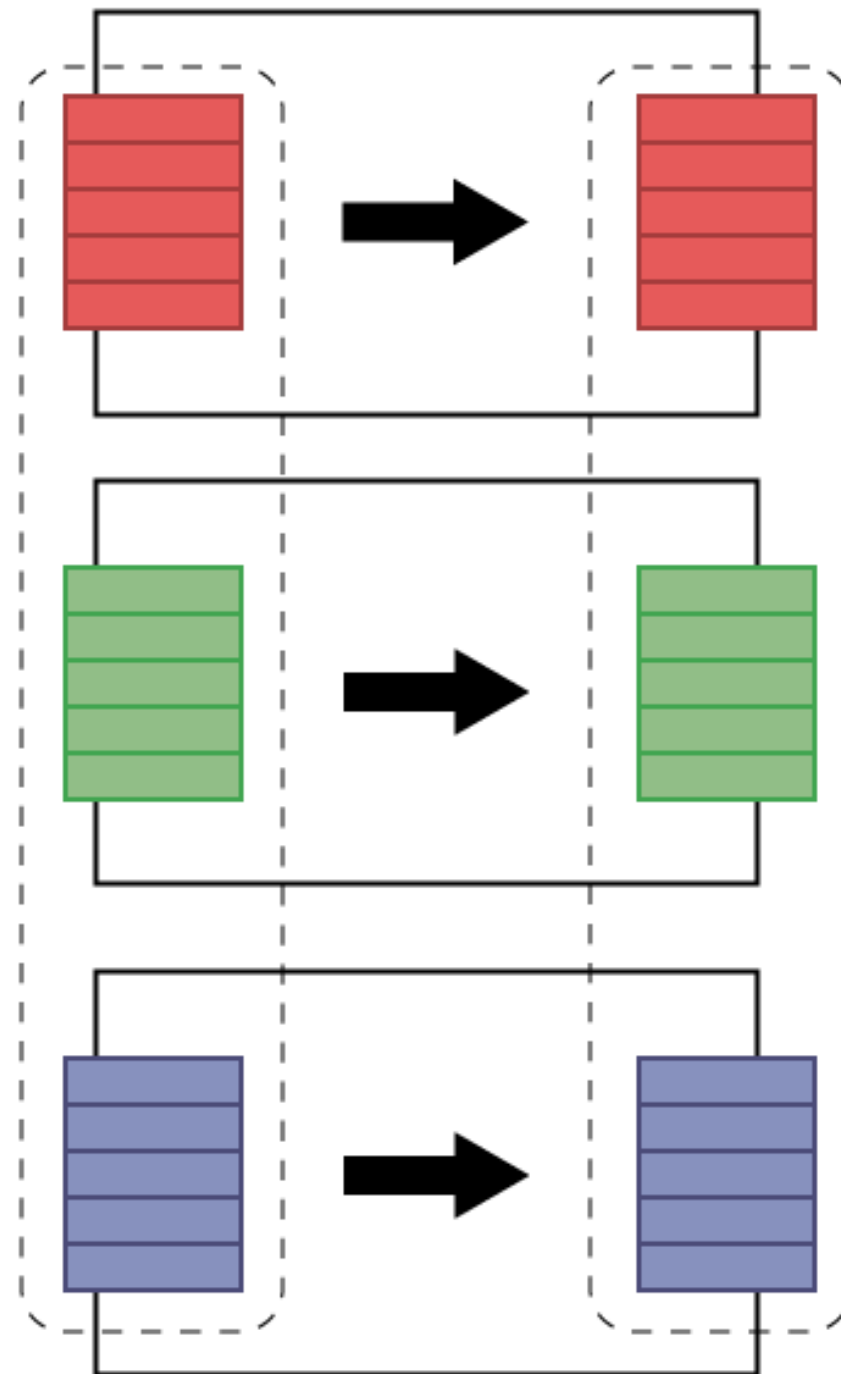
Les Stages: Ce sont regroupements d'opérations (transformations et actions) en fonction de leur nature et leurs dépendances. On distingue deux types de transformations. Celles qui nécessitent une étape de Shuffling (exemple: `groupByKey()`) qui sont dites "Wide" et celles qui n'en nécessitent pas (exemple: `Filter()`) qui sont dites "Narrow". A chaque transformation nécessitant un shuffle, un nouveau Stage est créé. Un Stage est constitué de plusieurs "Task".

FONCTIONNEMENT INTERNE D'UN RDD: CONCEPTS CLES

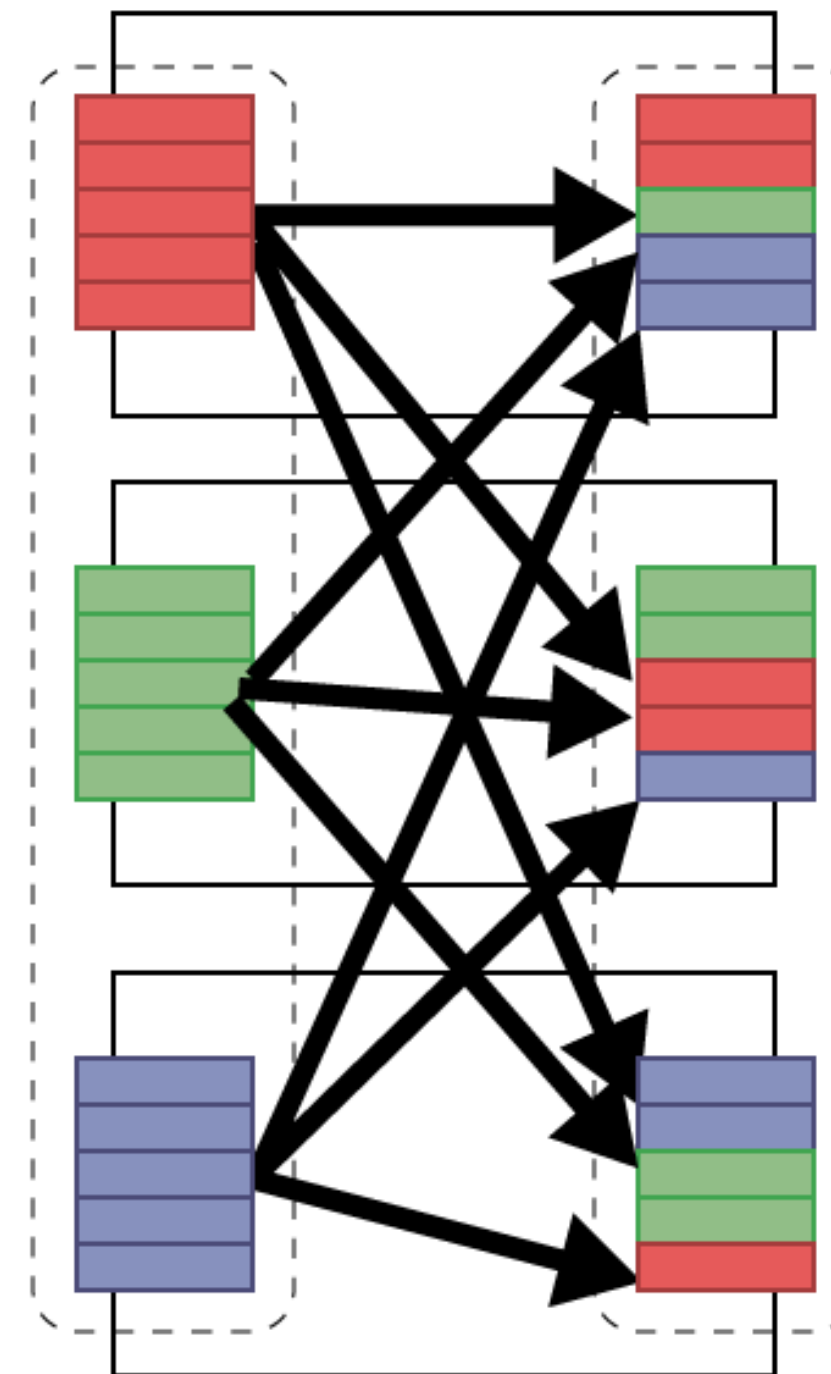
Wide & Narrow dependencies in Apache Spark



Narrow Transformation



Wide Transformation

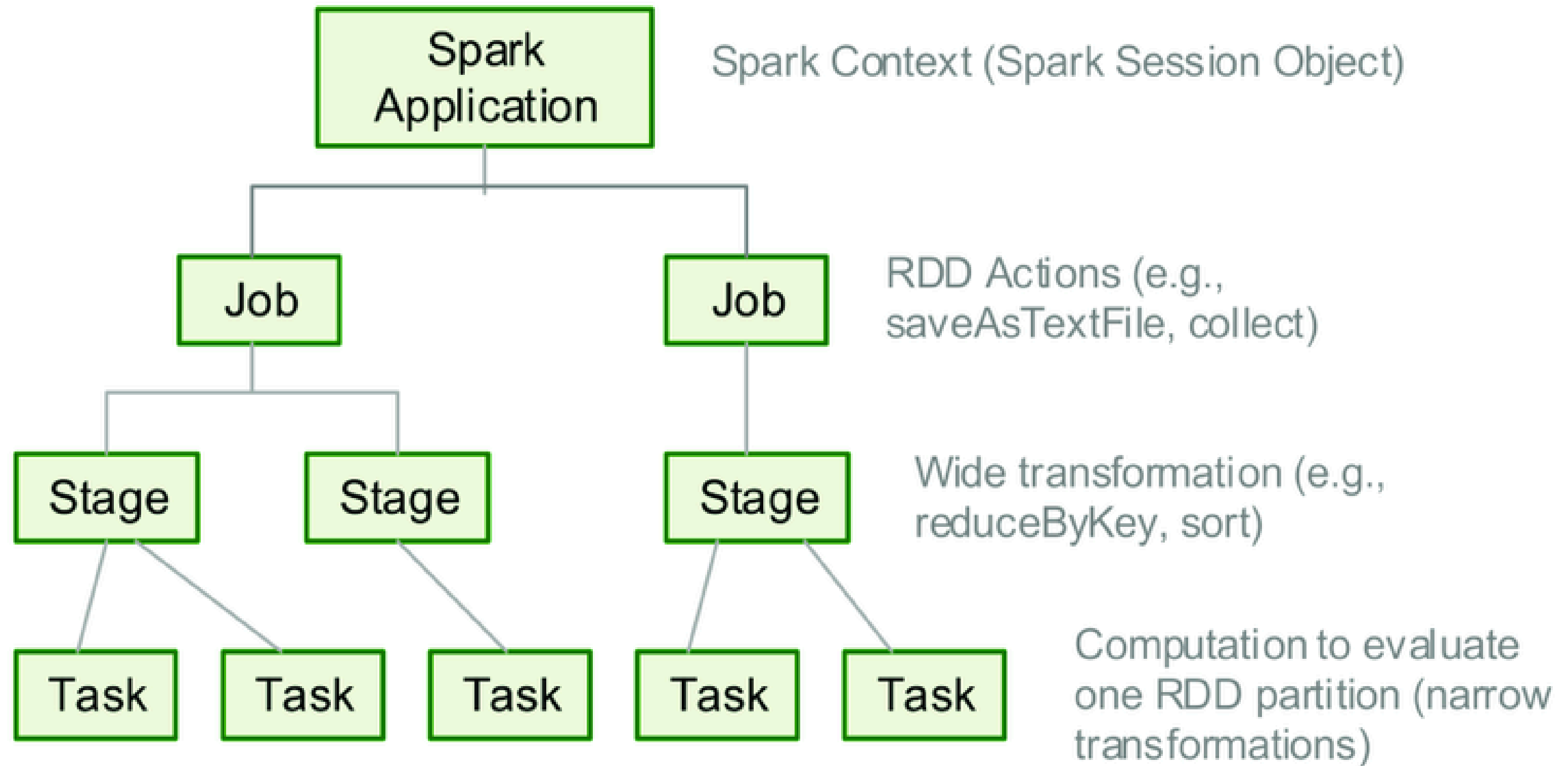


Les Tasks: C'est la plus petite unité de travail de Spark. C'est un traitement qui est exécuté sur une seule machine (exécuteur). Du point de vue pratique ce sont des opérations de transformations encapsulées pour être traitées dans des exécuteurs sur une partition de données

De façon sommaire, une application Spark est subdivisée en plusieurs Jobs en fonction des actions. Chaque Job peut être subdivisé en plusieurs Stages selon les types de transformations. Les Stages sont subdivisés en plusieurs Tasks qui sont exécutées en parallèle par des exécuteurs. Les exécuteurs sont des programmes qui tournent dans des Workers nodes (machines en réseau). Les Worker nodes sont orchestrés par un Cluster Manager.

FONCTIONNEMENT INTERNE D'UN RDD:

CONCEPTS CLES



```
import org.apache.spark.sql.Session

// Create a Spark Session
val spark = Session.builder
  .appName("Spark Job Stage Task Example")
  .getOrCreate()

val data = spark.read.option("header", "true").csv("path/to/your/file.csv")

val newData = data.withColumn("new_column", data("existing_column") * 2)

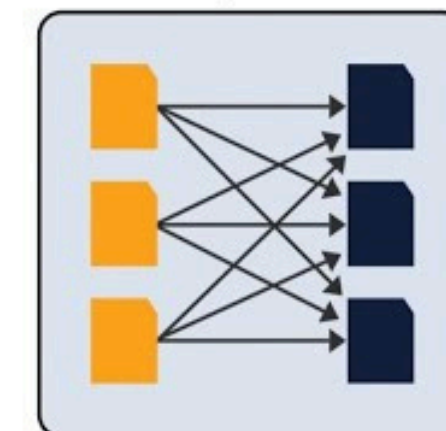
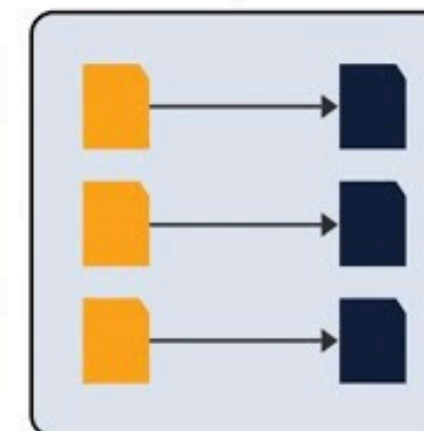
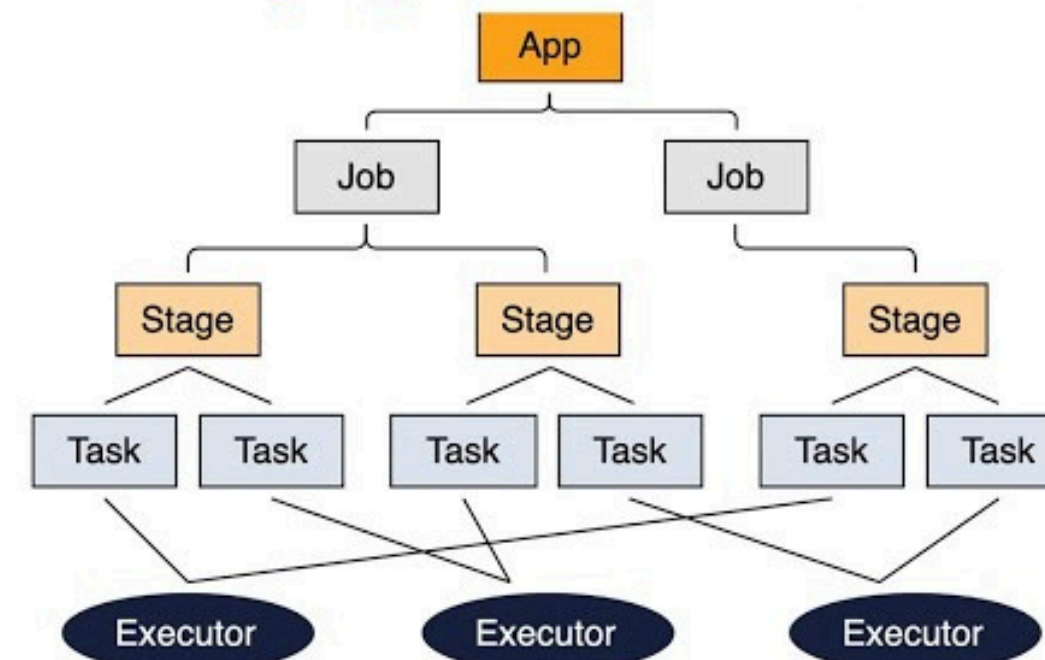
val result = newData.count()

println(result)

spark.stop()
```

The diagram illustrates the Spark architecture components and their interactions:

- Client:** An orange box at the top left. It has a dashed arrow labeled "Submit App (mode=cluster)" pointing to the Spark Master and another dashed arrow labeled "Application" pointing to the Driver.
- Spark Master:** A purple box. It is connected to the Client and the Spark Workers.
- Spark Workers:** Four blue boxes arranged horizontally. They are connected to the Spark Master and the Data Nodes.
- Driver:** An orange box within a dashed yellow border. It is connected to the Client and the Executors.
- Executors:** Three orange boxes within a dashed yellow border. They are connected to the Driver and the Spark Workers.
- Data Nodes:** Four blue boxes arranged horizontally. They are connected to the Spark Workers and the HDFS.
- HDFS:** A vertical stack of four colored boxes (A, B, C, D) on the left. A label "/large/file" is next to it. The Data Nodes are connected to the HDFS.
- RF 3:** A label indicating the replication factor for the HDFS data.
- DN + Spark:** A label at the bottom right indicating the combination of Data Nodes and Spark Workers.



Les DAG: Dans Apache Spark, DAG signifie Directed Acyclic Graph (graphe acyclique dirigé). Il s'agit d'un concept fondamental utilisé par le moteur d'exécution de Spark pour représenter et optimiser le flux d'opérations dans un travail de traitement de données. Il peut déterminer les dépendances et exécuter les étapes dans l'ordre le plus efficace.

En d'autres termes ce sont des séquences de transformations qui se terminent par une action.

Les DAGs dans la vie courante: Quelle est votre routine pour aller au travail ou en cours ?

```
rdd = sc.textFile("fichier.txt")
```

```
rdd2 = rdd.filter(lambda x: "Spark" in x)
```

```
rdd3 = rdd2.map(lambda x: x.upper())
```

```
rdd3.collect()
```

Quand Spark exécute une application il construit un DAG associé systématiquement en arrière plan avant de rencontrer une action qui déclenchera les opérations. L'objectif est :

- Savoir dans quel ordre exécuter les opérations.
- Optimiser l'exécution.
- Gérer la tolérance aux pannes (rejouer des étapes si une machine tombe).

FONCTIONNEMENT INTERNE D'UN RDD:

CONCEPTS CLES

```
rdd = sc.textFile("fichier.txt")           # Lecture = Stage 1  
  
rdd2 = rdd.map(lambda x: x.split())        # Toujours Stage 1  
  
rdd3 = rdd2.groupByKey()                   # Besoin d'un shuffle = Nouveau Stage 2  
  
rdd3.collect()                             # Action → déclenche tout
```

SPARK SQL est le composant de SPARK qui permet de manipuler les données sous forme tabulaire en s'appuyant sur SQL.

Les avantages sont les suivants:

- Facilité d'utilisation: Le SQL est accessible à tous
- Optimisation dans les traitements: Spark possède un composant pour réduire l'utilisation des ressources et améliorer la rapidité d'exécution des requêtes
- Intéropérabilité: Il permet de manipuler des données de plusieurs formats
- Intégration: Il est compatible avec JAVA, R, SCALA, PYTHON

Pour manipuler des dataframes il faut instantier une session SPARK dans la même dynamique qu'on crée un SPARK CONTEXT pour manipuler des RDDs. La différence majeure réside sur le type de données manipulées.

La commande:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("DataFrameExample").getOrCreate()
```

ou

```
warehouse_location = abspath('spark-warehouse')
```

```
spark = SparkSession.builder.appName("Python Spark SQL Hive integration example")
```

```
.config("spark.sql.warehouse.dir", warehouse_location).enableHiveSupport().getOrCreate()
```

Lorsque les données sont chargées dans un DataFrame on peut créer des vues (en approche datamart) au sein SPARK. Ces vues sont des tables HIVE. En effet SPARK contient un composant minimaliste (avec le moins de dépendance possible) HIVE qui permet de manipuler des données avec une approche SQL (si vous activez Hive).

Pour manipuler les données sous format SQL il existe 2 approches:

- createOrReplaceTempView: pour créer des vues temporaires qui disparaissent après la fermeture de la session.

```
df.createOrReplaceTempView("employees")
```

- saveAsTable : pour sauvegarder les tables sur le disque.

```
df.write.format("parquet").mode("append").saveAsTable("my_table")
```

Dans certains il est pertinent de stocker les résultats intermédiaires des traitements dans le but d'optimiser les performances de l'application SPARK.

On distingue 2 façons de persistance des données:

- Cache: Cette opération permet le stockage en mémoire des données
- Persist: Cette opération permet le stockage sur disque des données et en mémoire en fonction du paramètre passé à la fonction.
 - MEMORY_ONLY: Stockage en mémoire
 - MEMORY_ONLY_SER: Stockage en mémoire désérialiser pour réduire l'espace sur disque
 - MEMORY_AND_DISK: Stockage en mémoire et sur disque
 - MEMORY_AND_DISK_SER: Stockage en mémoire et sur disque de façon désérialisée

Il existe 2 modes de déploiement d'une application Spark:

- Client: le driver reste local, utile pour tester ou déboguer.
- Cluster: le driver est dans le cluster YARN, plus robuste et idéal en production.

Pour soumettre une application spark la syntaxe est la suivante:

*spark-submit *

*--master yarn *

*--deploy-mode cluster *

*--executor-memory 2G *

*--num-executors 2 *

--executor-cores 2 fichier.py

--master yarn/spark/mezos/kubernetes : Mode de déploiement sur YARN

--deploy-mode cluster/client: Où tourne le driver

--executor-memory: Mémoire pour chaque exécuteur

--num-executors: Nombre total d'exécuteurs

--executor-cores: Nombre de cœurs par exécuteur

4Go / 8 machines => 500Mb / machine

5 blocs HDFS de fichiers pour 500mb (avec la valeur de 128 Mb de HDFS)

1Go => 1000Mb

executor memory = Taille de chaque bloc majoré de 25%

nombre total de mémoire possible = taille du cluster

Vous êtes dans la peau d'un data engineer et vous devez mettre en place une architecture Lakehouse pour répondre à une problématique business de votre choix.

- 1- Chercher votre problématique business à résoudre
- 2- Proposer des sources de données open source (structurées et sémi structurées)
- 3- Proposer les traitements que vous pouvez faire entre les différentes couches bronze-> silver et silver->gold
- 4- Implémenter la solution associée à la problématique étudiée avec Hadoop, pySpark et Spark SQL

- Les types de données et le nombre de sources de données
- Le bon usage du LakeHouse pour stocker les données aux bons endroits au format adapté
- La complexité des traitements effectués entre les couches du Lakehouse
- La pertinence business de la solution apportée
- La qualité de la diapositive
- La clarté de l'architecture utilisée
- Le délai de restitution du travail
- Les captures d'écran des outils Yarn(ResourceManager) , NodeManager et Spark UI
- Utilisation d'un datawarehouse comme Hive (couche silver), utilisation d'une base de données comme datamart